SYSTEM DOCUMENTATION

# PryingEyes

Privacy-first contact sharing — setup guide, use-case model and technical reference for the Phase 1 website

| | |
|---|---|
| **Document** | System documentation and use-case model |
| **System version** | Phase 1 MVP website and phone app (`pryingeyes 0.10.1`; documentation v1.14) |
| **Based on** | Development Agent Prompt v1.1; Product Requirements and Security Specification v1.0 |
| **Owner** | Johnson Sikhumbuzo Dlamini |
| **Date** | 4 November 2026 |
| **Classification** | Internal / Confidential |


**1–4** Overview, architecture, setup and configuration

**5–6** Use-case diagrams and specifications

**7–8** Key flows, operations and troubleshooting

**9** API, pages, scripts, data and glossary

# Contents


**1 Introduction** **1**

1.1 Purpose of this document . . . 1
1.2 What PryingEyes does . . . 1
1.3 Protection tiers . . . 2
1.4 Token classes . . . 2
1.5 Scope of Phase 1 . . . 3
1.6 Related documents in the repository . . . 3

**2 Architecture** **4**

2.1 Components . . . 4
2.2 Technology stack . . . 5
2.3 Repository layout . . . 5
2.4 Data model . . . 6
2.5 Keys and cryptography . . . 7
2.6 Audit chain . . . 8

**3 Installation and setup** **9**

3.1 Requirements . . . 9
3.2 Install and run PryingEyes . . . 9
3.3 Development tools . . . 10
3.3.1 Seeded accounts . . . 10
3.4 First walkthrough . . . 11
3.5 Sending real emails with Resend (optional) . . . 11
3.6 Production-style local run . . . 12
3.7 Running everything in containers . . . 12
3.8 Updating to a new version . . . 12
3.9 Resetting the local environment . . . 12

**4 Configuration reference** **13**

4.1 Environment modes . . . 13
4.2 Core settings . . . 14

4.3 Secrets . . . 15
4.4 Providers . . . 15
4.5 Policy defaults . . . 16

**5 Use-case model** **18**
5.1 Notation . . . 18
5.2 Actors . . . 18
5.3 System context . . . 20
5.4 Accounts and authentication . . . 21
5.5 Numbers and tokens . . . 22
5.6 Decode gate and contact . . . 23
5.7 Chat and safety . . . 24
5.8 Audit, evidence and data rights . . . 25
5.9 Administration and housekeeping . . . 26

**6 Use-case specifications** **27**
6.1 Actor and use-case matrix . . . 27
6.2 Accounts and authentication . . . 28
6.3 Numbers and tokens . . . 31
6.4 Decode gate and contact . . . 33
6.5 Chat and safety . . . 36
6.6 Audit, evidence and data rights . . . 37
6.7 Administration and operations . . . 39

**7 Key flows and lifecycles** **41**
7.1 Decode gate decision flow . . . 41
7.2 How people chat and call privately . . . 42
7.3 In-app call lifecycle . . . 43
7.4 Native app contract . . . 44
7.5 A complete Tier 3 contact . . . 45
7.6 Token lifecycle . . . 46
7.7 Gate request lifecycle . . . 46

**8 Phone app** **47**
8.1 Running it . . . 47
8.2 How the app authenticates . . . 48
8.3 Calls in the app . . . 48

8.4 Checks . . . 48

**9 Safe groups and statuses 49**

9.1 What people can do . . . 49
9.2 The contact-details guard . . . 50
9.3 Contact codes on posts . . . 50
9.4 Links that open the app . . . 50
9.5 API . . . 51
9.6 Codes without a phone number . . . 52
9.7 Codes shared anywhere . . . 52

**10 The system's screens 53**

10.1 Look and feel . . . 54
10.2 Website . . . 54
10.3 Phone app . . . 54

**11 Operations, testing and troubleshooting 65**

11.1 Automated tests and acceptance checks . . . 65
11.2 Running in production . . . 66
11.3 Email with Resend . . . 66
11.4 Data retention schedule . . . 67
11.5 Backups and recovery . . . 67
11.6 Monitoring . . . 67
11.7 Troubleshooting . . . 68

**12 Reference 70**

12.1 HTTP API . . . 70
12.1.1 Gate response statuses . . . 72
12.1.2 Common error codes . . . 72
12.2 Pages . . . 73
12.3 Command-line scripts . . . 74
12.4 Notifications . . . 75
12.5 Glossary . . . 75

**A Design decisions 77**

# List of Figures


2.1 Component architecture . . . . . 4

2.2 Main entities (arrows point to the referenced table; * = many) . . . . . 6

5.1 System context: actors and subsystems . . . . . 20

5.2 Use cases — accounts and authentication . . . . . 21

5.3 Use cases — numbers and tokens . . . . . 22

5.4 Use cases — decode gate and contact . . . . . 23

5.5 Use cases — chat and safety . . . . . 24

5.6 Use cases — audit, evidence and data rights . . . . . 25

5.7 Use cases — administration and housekeeping . . . . . 26

7.1 Decode gate decision flow . . . . . 41

7.2 In-app call states. *Missed*, *busy*, and *cancelled* while the owner's app was closed all notify the owner. . . . . 43

7.3 An answered in-app call. The API never sees audio; TURN forwards packets it cannot decrypt, and each side only sees the relay's address. . . . . 43

7.4 Sequence — Tier 3 request approved, then an in-app call . . . . . 45

7.5 Token states . . . . . 46

7.6 Gate request stages . . . . . 46

10.1 Home page: share a code, not your phone number . . . . . 55

10.2 Sign-up: email and password, with a generated handle (nobody sees the email) . . 55

10.3 New code: a name, where it will be shared, how careful to be, and how long it works 56

10.4 A new code, ready to share as a link, a short code or a QR code . . . . . 56

10.5 A caller opens a code while signed out: create an account or sign in, then come straight back to it . . . . . 57

10.6 The gate after the human check: message or call the owner in the app . . . . . 57

10.7 Private chat. The owner has shared a one-person code, shown as **Open contact code**; tapping it opens that code's gate . . . . . 58

10.8 An in-app call ringing: over the internet, through the TURN relay, with no phone numbers . . . . . 58

10.9 Tier 3: the owner sees who is asking, how they were checked, and their note, then decides . . . . . 59

10.10A group as its admin sees it: a post with a contact code, a phone number refused by the guard, and someone asking to join . . . . . 59

10.11Groups: 24-hour statuses, finding a group to ask to join, and your groups . . . . . 60

10.12Settings: account, the optional **Show my number**, two-step sign-in and signed-in devices . . . . . 60

10.13Phone app: getting in . . . . . 61

10.14Phone app: reaching someone, and your own codes . . . . . 62

10.15Phone app: making and sharing a code . . . . . 63

10.16Phone app: groups . . . . . 64

# List of Tables


1.1 Protection tiers . . . . . 2
1.2 Token classes . . . . . 2

2.1 Technology stack . . . . . 5
2.2 All database tables . . . . . 7
2.3 Secrets and what they protect . . . . . 7

3.1 Software requirements . . . . . 9
3.2 Where to see what the system would have sent . . . . . 10

4.1 What `APP_ENV` allows . . . . . 13
4.2 Core settings . . . . . 14
4.3 Provider settings . . . . . 15
4.4 Gate budgets, lockouts and caps . . . . . 16

5.1 Actors . . . . . 19

6.1 Which actor takes part in which use case (● primary, ❍ secondary) . . . . . 27

11.1 Test suites and acceptance checks . . . . . 65
11.2 Retention, applied by the housekeeping worker . . . . . 67
11.3 Common problems . . . . . 68

12.1 API endpoints . . . . . 70
12.3 Web pages . . . . . 73
12.4 npm scripts . . . . . 74

CHAPTER 1

# Introduction

## 1.1 Purpose of this document

This document is the long-term reference for the PryingEyes Phase 1 website. It explains:

- what the system does and how it is built (Chapters 1–2);
- how to install, configure and run it on an Ubuntu desktop or server (Chapters 3–4);
- every use case, as UML diagrams and as written specifications (Chapters 5–6);
- the main runtime flows, operations, testing and troubleshooting (Chapters 7–11);
- a reference for the API, pages, scripts, data retention and terms (Chapter 12).

It is written for developers maintaining the code, operators running it, reviewers and penetration testers assessing it, and supervisors or funders who want to understand what it does. It describes the system as built, matching the code in the `pryingeyes` repository at version 0.1.0.

## 1.2 What PryingEyes does

People share phone numbers on social media, classifieds, dating apps and community groups, where scrapers, scammers and stalkers can collect them. PryingEyes lets an *owner* post a short code instead of their number. Anyone who wants to reach them, a *caller*, must pass through the platform's **decode gate**. The gate is verified, rate-limited and fully logged. Depending on the protection tier the owner chose, it then:

- shows the number as a watermarked image;
- connects an **in-app call** over data, in which neither side sees the other's number;
- asks the owner for consent first; or
- opens an in-app **chat**.

Three design decisions carry most of the security:

1. **Opaque random tokens.** A code is random (80 bits for links, 50 bits for short codes) and contains nothing about the number. The link between a code and a number exists only inside the server, so a code can't be decoded offline.
2. **A gated, audited decode step.** Nobody gets a number without an account, a human check and server-side budgets. Some tiers add a one-time code and the owner's consent. Every attempt is written to a tamper-evident log the owner can see and export as signed evidence.
3. **No plaintext numbers anywhere.** Numbers are encrypted per user (AES-256-GCM, envelope encryption). Codes are stored only as keyed fingerprints. Logs are filtered so they can't contain anything shaped like a phone number.

## 1.3 Protection tiers

Owners choose a tier per token, so one person can run a public business line and a personal-safety code at the same time.

**Table 1.1:** Protection tiers

| Tier | What the caller must pass | What the caller can get | Recommended for |
|---|---|---|---|
| T1 Open | Sign-in, human check, budgets | Chat or in-app call; a watermarked reveal if the code has a number | Business and public lines |
| T2 Gated | Tier 1 + a one-time code emailed to the caller; at most 3 Tier 2 contacts per caller per day | Chat, in-app call, or a reveal if the code has a number | Classifieds, community groups |
| T3 Consent | Tier 2 + the owner's approval (24-hour window), optionally with a 280-character note | Chat and in-app call; one reveal only if the code has a number and the owner chooses *Reveal once* | Marketplaces, dating, new contacts |
| T4 Vault | *Phase 2.* Shown in the interface as "Coming soon"; can't be selected. | | |

When an owner creates a token, the interface recommends a tier from where they plan to post it:

- business → Tier 1;
- classifieds or community → Tier 2;
- marketplace, dating or someone new → Tier 3, with a 48-hour expiring token.

## 1.4 Token classes

**Table 1.2:** Token classes

| Class | Lifetime | Typical use |
|---|---|---|
| Standard | Until revoked or rotated | Business line, long-term community contact |
| Expiring | Set time-to-live, default 48 hours; the owner can extend it | Classified adverts, short campaigns |
| Disposable | One delivered outcome, then *consumed* | A first contact, a one-off meeting |
| Purpose-bound | Owner label and optional expiry; rotation reminder every 90 days | A marketplace persona; work versus personal |

Each protected number can have at most 10 live tokens, and each account can protect up to 5 numbers. A link looks like `https://<host>/t/K7QM-4XR2-9BDN-2F9X`; a short code looks like `RW4TP-8NMKD`. Short codes are for platforms that don't allow links.

## 1.5 Scope of Phase 1

**In scope (built and tested)**

- Accounts: email sign-up, generated handles
- Passkeys, authenticator apps, recovery codes
- Encrypted number vault
- Four token classes; Tiers 1–3
- Watermarked reveal, in-app calling, chat
- Owner dashboard and audit feed
- Signed evidence packs; panic revoke-all
- Reports and moderation
- Data export and erasure; retention jobs
- Dual-control administration

**Out of scope (Phase 2 and later)**

- Tier 4 Vault
- End-to-end encrypted chat (a seam is left in the chat module)
- Installable web app and native apps
- Partner API, browser extension
- Decoy tokens, breach monitoring
- Trusted-contact escalation
- Virtual per-context numbers
- Chat attachments, call recording

## 1.6 Related documents in the repository

| | |
|---|---|
| `README.md` | Quick start for developers. |
| `docs/THREAT_MODEL.md` | Threats T1–T14 with the control and test for each. |
| `docs/ASSUMPTIONS.md` | Every decision the specification left open (summarised in Appendix A). |
| `docs/DEPLOY.md` | Server deployment guide, including nginx and write-once anchors. |
| `docs/LAUNCH_CHECKLIST.md` | Blockers before public launch: legal review, penetration test, KMS. |
| `docs/SECURITY_REVIEW.md` | Builder's self-review for penetration testers. |
| `docs/FINAL_REPORT.md` | Milestone reports and verification results. |

CHAPTER 2

# Architecture

## 2.1 Components

PryingEyes is one Next.js application, with its business logic in a TypeScript library, backed by PostgreSQL and Redis. A separate worker process runs scheduled housekeeping. External services handle email (Resend) and the human check (Turnstile); PryingEyes sends no text messages and places no phone calls. Each one sits behind an adapter, and each adapter has a development stand-in, so the whole system runs offline on a laptop.

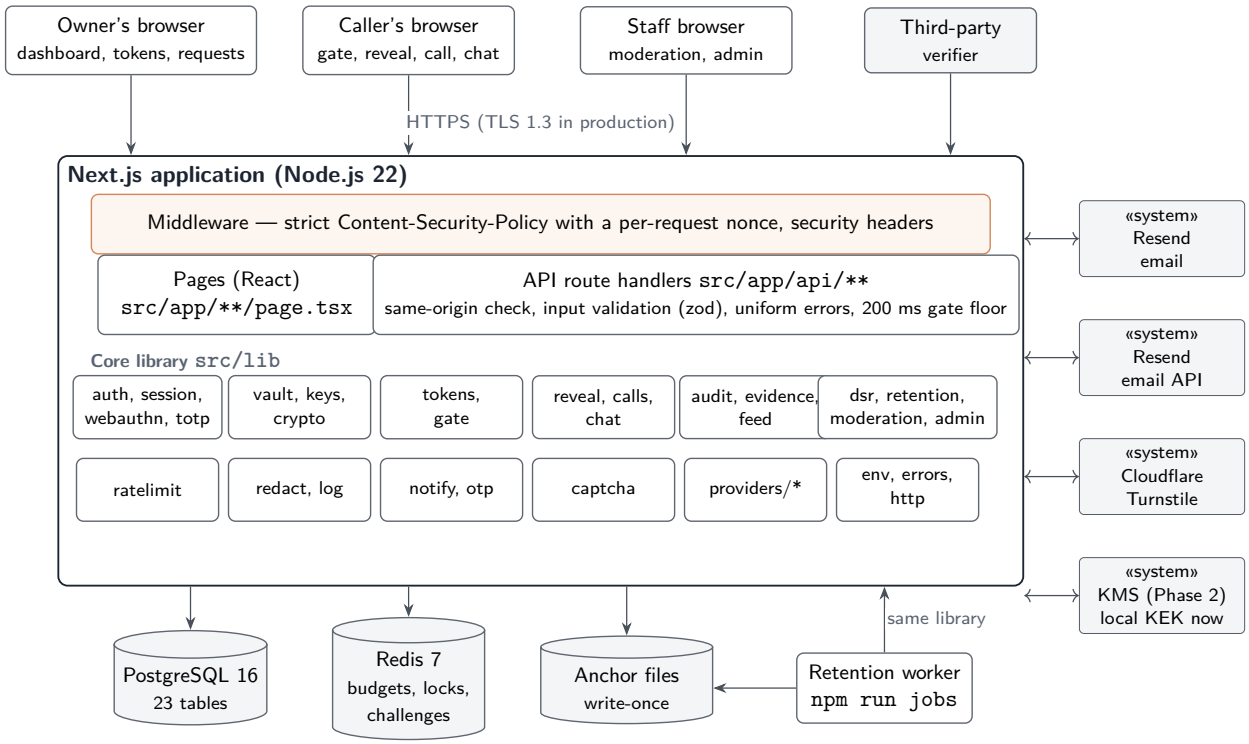



**Figure 2.1:** Component architecture

Route handlers are thin. They authenticate, validate input and call one function in `src/lib`, which holds all the business rules. The same library is used by the retention worker and the command-line scripts, so behaviour is identical wherever it runs.

## 2.2 Technology stack

**Table 2.1:** Technology stack

| Concern | Choice | Notes |
|---|---|---|
| Runtime | Node.js 22 | Next.js 15 (App Router), React 19, TypeScript (strict) |
| Database | PostgreSQL 16 | Drizzle ORM; plain SQL migrations in `drizzle/` |
| Cache and counters | Redis 7 | Rate budgets, lockouts, WebAuthn challenges, pending phone numbers |
| Passwords | Argon2id | `@node-rs/argon2`; 19 MiB, 2 iterations, parallelism 1 |
| Passkeys | WebAuthn | `@simplewebauthn`; user verification required |
| Authenticator apps | TOTP (RFC 6238) | Built in; replay-protected |
| Encryption | AES-256-GCM | Per-user data keys wrapped by a key-encryption key (KEK) |
| Signatures | Ed25519 | Evidence packs |
| Reveal images | `@napi-rs/canvas` | Server-rendered PNG with a diagonal watermark |
| Evidence PDF | `pdf-lib` | |
| Phone numbers | `libphonenumber-js` | Normalised to E.164; default country Eswatini (SZ) |
| Logging | `pino` | Every field passes through the redactor |
| Tests | Vitest, Playwright, axe | Integration tests against real PostgreSQL and Redis |
| Email | Resend (REST API) | Plain text + HTML; idempotent retries; development outbox locally |
| Containers | Docker Compose | PostgreSQL, Redis; optional app and worker images |

## 2.3 Repository layout

```
pryingeyes/
  src/
    app/            pages (React) and api/ route handlers
    components/     shared UI: Gate, HandleField, StepUp dialog, Captcha, ...
    lib/            security core and business rules (see Figure 2.1)
    db/             schema.ts (Drizzle) and the connection pool
    i18n/           interface strings (English; other languages can be added)
    middleware.ts   CSP and security headers
  drizzle/          SQL migrations (0000_init, 0001_audit_append_only)
  scripts/          migrate, keys, seed, jobs, attack, scan, verify, load test
  tests/            unit/, integration/ (real DB), e2e/ (Playwright)
  docs/             markdown docs and this LaTeX documentation (docs/latex)
  assets/fonts/     font used to render reveal images
  docker-compose.yml, Dockerfile, .env.example, package.json
```

## 2.4 Data model

Figure 2.2 shows the main entities and how they relate. All primary keys are UUIDs except where noted. Every table holding personal data cascades from `users`, so erasing an account removes it in one statement.

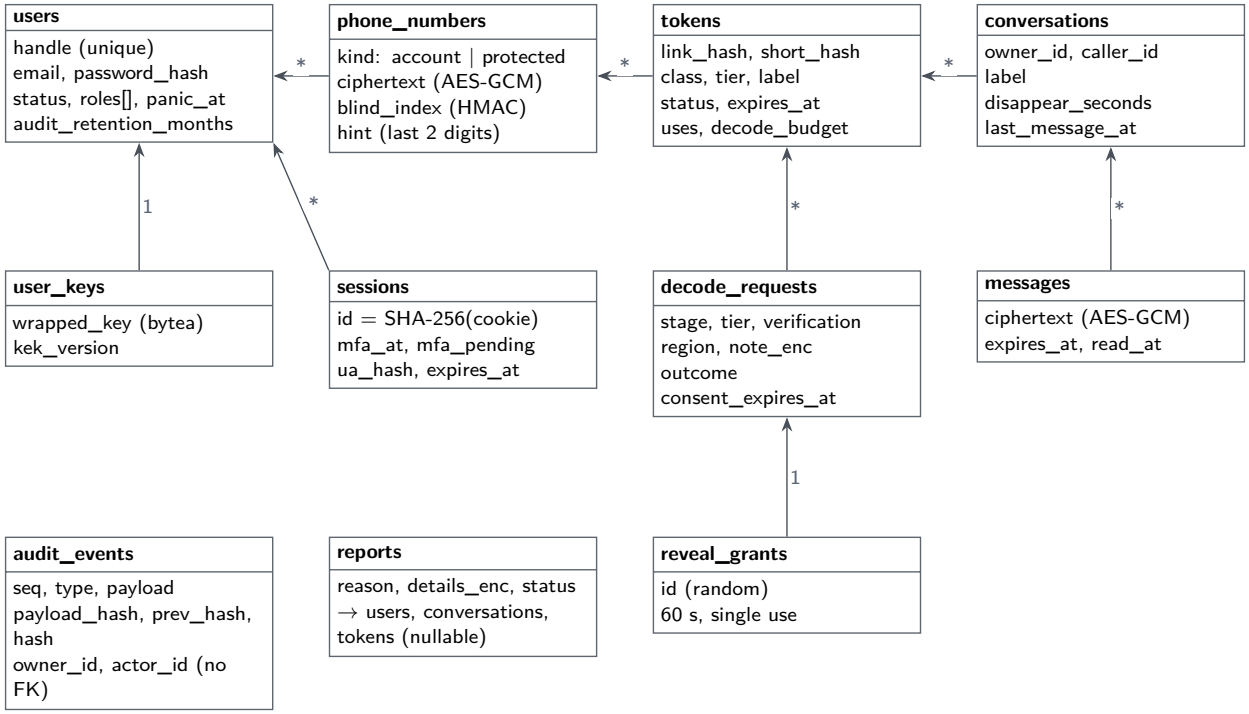



**Figure 2.2:** Main entities (arrows point to the referenced table; * = many)

**Table 2.2:** All database tables

| Table | Holds |
|---|---|
| `users`, `user_keys` | Accounts; each user's wrapped data key |
| `phone_numbers` | Encrypted numbers: one *account* phone for sign-in, plus optional numbers an owner lets approved people see |
| `passkeys`, `totp_secrets`, `recovery_codes` | Second factors (TOTP secret encrypted; recovery codes as keyed hashes) |
| `sessions`, `account_recoveries` | Sign-in sessions; open recoveries and their cooldown |
| `verification_tokens`, `otp_challenges` | Email links and one-time codes, stored as hashes |
| `tokens`, `decode_requests`, `reveal_grants` | Codes (as fingerprints), gate requests and one-time reveal grants |
| `conversations`, `messages`, `blocks` | Chats (messages encrypted) and blocks |
| `reports`, `notifications` | Abuse reports (details encrypted) and in-app notifications |
| `audit_events`, `audit_anchors` | Hash-chained audit log; daily head hashes |
| `admin_actions` | Dual-control break-glass requests |
| `dev_outbox` | Development only: captured emails |

## 2.5 Keys and cryptography

**Table 2.3:** Secrets and what they protect

| Setting | Purpose |
|---|---|
| `MASTER_KEK` | Key-encryption key. Wraps each user's 256-bit data key, which encrypts that user's numbers, TOTP secret, intro notes and chat keys. Rotate by adding a new version and moving the old one to `KEK_PREVIOUS`, then run the *re-wrap keys* admin action. |
| `TOKEN_PEPPER` | HMAC key for token fingerprints: `HMAC-SHA256(code, pepper)`. One indexed lookup, constant-time compare. |
| `BLIND_INDEX_KEY` | HMAC key for the phone-number blind index (de-duplication without decrypting). |
| `SECRETS_HMAC_KEY` | Hashes for one-time codes and recovery codes, user-agent binding, and the moderation data key. |
| `EVIDENCE_SIGNING_KEY` | Ed25519 private key (PKCS#8) for signing evidence packs. The public key is at `/api/evidence/public-key`. |

Ciphertexts are bound to their row with associated data. For example, a phone number is encrypted with `phone:<row id>` as associated data, so a ciphertext copied into another row fails to decrypt. Only `vault.decryptNumberFor` decrypts numbers, and only for two purposes: reveal and data export.

## 2.6 Audit chain

Every security-relevant action appends a row to `audit_events` inside the same database transaction. Each row stores:

$$\text{hash}_n = \text{SHA-256}(\text{prev_hash} \,|\, \text{seq} \,|\, \text{at} \,|\, \text{type} \,|\, \text{actor} \,|\, \text{owner} \,|\, \text{subject} \,|\, \text{payload_hash})$$

A database trigger forbids `UPDATE` and `DELETE` on the table. The one exception is setting a payload to `NULL`, which erasure and retention need. Because each link hashes the payload's *hash*, the chain still verifies after personal data is removed. Each completed day's head hash is written to `audit_anchors` and to a write-once file in `AUDIT_ANCHOR_DIR`.

CHAPTER 3

# Installation and setup

This chapter sets up a complete local copy on Ubuntu (22.04 or 24.04), Windows or macOS. Every email is captured locally instead, so no accounts with external providers are needed. Since version 0.10 the website is self-contained. It needs only Node.js: its PostgreSQL database and its cache are built in, and it prepares itself the first time it starts. Allow about 5 minutes.

## 3.1 Requirements

**Table 3.1:** Software requirements

| Software | Version | Check with |
|---|---|---|
| Node.js and npm | 22.x (20.11+ works) | `node -v` |
| Git or unzip | any | |
| A browser | recent Chrome, Edge, Firefox or Safari | needed for passkeys |
| Free port | 3000 | |
| coturn (TURN relay) | optional | only for in-app calls; `sudo apt install coturn` or `docker compose up -d coturn` |
| Resend account | optional for local use | needed to send real emails |

## 3.2 Install and run PryingEyes

```
unzip pryingeyes.zip && cd pryingeyes
npm install
npm run dev                    # http://localhost:3000
```

Open http://localhost:3000. The first `npm run dev` does the setup itself:

1. **Settings.** `scripts/prepare.mjs` creates `.env` from `.env.example` with this computer's address (`http://localhost:3000`) and fills in fresh secret keys, with file mode 0600. It never replaces an existing key. If `.env` or its keys are missing but the built-in database already holds data, it refuses to start: data encrypted with the old keys can't be read with new ones.
2. **Database.** With `DATABASE_URL` empty, PostgreSQL runs inside the website (PGlite, PostgreSQL 18 compiled to WebAssembly). Its files are in `var/data/postgres`. When the server starts, `src/instrumentation.ts` applies every migration in `drizzle/`.
3. **Cache.** With `REDIS_URL` empty, rate limits, call signalling and passkey challenges are held in memory (`src/lib/memory-redis.ts`).
4. **Background jobs.** With `BACKGROUND_JOBS=on`, the website runs the retention pass every 10

minutes and the missed-call sweep every 10 seconds itself. There is no separate worker.

> **One process owns the built-in database**
>
> The built-in database is a folder of files, not a server, so only the website may open it. A lock file (`var/data/postgres.lock`) makes scripts such as `npm run seed` or `npm run admin:create` refuse while the website runs; stop it first. Set `DATABASE_URL` and `REDIS_URL` to use PostgreSQL and Redis servers instead, for example to run several copies of the app. The schema and every feature are identical, and the server test suite passes on both setups.

To add demo accounts, stop the website and run `npm run seed`; it refuses unless `APP_ENV=development`. Your own account can become an admin without any script: put its email in `ADMIN_EMAILS`, and it gets the role once the email is confirmed.

**The Docker route.** `start.sh` and `docker-compose.yml` still run PostgreSQL, Redis and coturn in containers (`./start.sh setup`, then `./start.sh web`), and they drive the phone-app modes. They are optional.

## 3.3 Development tools

**Table 3.2:** Where to see what the system would have sent

| Tool | Use |
|---|---|
| `http://localhost:3000/dev/outbox` | Every email (confirmation links, codes, alerts), newest first. Email confirmation links have an **Open link** button. |
| `npm run email:test -- --to=<you>` | Sends one test email through the configured provider; use it to check your Resend setup. |
| `var/seed-credentials.txt` | Seeded accounts, their authenticator keys and the demo token links. |
| `npm run dev:totp -- <key>` | Prints the current 6-digit code for an authenticator key (for seeded accounts). |
| `npm run jobs:once` | Runs one housekeeping pass immediately. |

### 3.3.1 Seeded accounts

`npm run seed` creates the accounts below, all with the password `pryingeyes demo password` and an authenticator app already enrolled. The authenticator keys are printed and saved in `var/seed-credentials.txt`.

| Email | Role | Has |
|---|---|---|
| `owner@demo.pryingeyes.local` | Owner | One protected number; Tier 1, 2 and 3 tokens |
| `moderator@demo.pryingeyes.local` | Moderator | Moderation queue |
| `admin1@demo.pryingeyes.local` | Administrator | Can request admin actions |
| `admin2@demo.pryingeyes.local` | Administrator | Can approve the first admin's requests |

To sign in as one of them, enter the email and password, then run `npm run dev:totp -- <their key>`

and type the code shown.

## 3.4 First walkthrough

1. **Become an owner.** Choose **Create account**, enter an email and password, keep the suggested handle (or tap **Suggest another**) and tick the 18+ box. Open the confirmation link from `/dev/outbox`, then sign in. Onboarding asks you to:
   - create a passkey, or use an authenticator app;
   - save your 10 recovery codes;
   - add the number to protect.
2. **Create a token.** Go to **+ New token**, pick where you'll post it (the tier is recommended for you), add a label and create it. Copy the link now: it is shown only once.
3. **Be a caller.** Open the link in a private window. Leave the country on **Eswatini (+268)** and enter any number such as `7612 3456` (no dialling code needed), read the code from `/dev/outbox`, keep the suggested handle and confirm you're 18+. Tick the human check.
   - **Tier 1:** choose **Show the number**. The watermarked image hides itself after 60 seconds.
   - **Tier 2:** first confirm a second code sent to your phone.
   - **Tier 3:** add a note and send the request. As the owner, open **Requests** and choose **Allow messages and calls**. As the caller, choose **Call in the app** or **Chat in the app**.
4. **Call in the app.** Keep the owner signed in in one browser window and the caller in another (a private window, or a second browser profile). Allow the microphone when asked. On the gate or in the chat, the caller chooses **Call in the app**. The owner's window shows a ringing banner on any page; choose **Answer**. Both screens reach **Connected** and the audio goes through the local coturn relay. To see a missed call, close the owner's window, call, and wait 45 seconds (or cancel): the owner's notifications and the chat show the missed call.
5. **Chat and call privately.** From the gate, choose **Chat in the app**. As the owner, the **Chats** menu item now shows an unread count and a "started a chat" notification arrives (in `/dev/outbox` too). As the caller, **Call in the app** in the chat rings the owner while the gate request is valid.
6. **Panic.** On the dashboard, choose **Panic: revoke all**. Every link stops working at once.
7. **Staff tools.** Run `npm run admin:create - -handle=<your handle> -role=moderator` (or `admin`), then reload. **Moderation** and **Admin** appear in the menu.

## 3.5 Sending real emails with Resend (optional)

Locally, emails are captured at `/dev/outbox` and nothing is sent. To receive real emails while testing:

1. Create a free account at `https://resend.com` and an API key with sending access.
2. In `.env`, set `EMAIL_PROVIDER=resend` and `RESEND_API_KEY=re_...`. Until you have verified a domain, leave `EMAIL_FROM` set to Resend's test sender, `onboarding@resend.dev`. Until then, Resend only delivers to the email address that owns the account, so sign up with that address.
3. Restart `npm run dev` and run `npm run email:test - -to=<your Resend account email>`.

In development, emails sent through Resend are still copied to `/dev/outbox`. To send to anyone, verify your own domain in Resend and change `EMAIL_FROM` (§11.3).

## 3.6 Production-style local run

The development server (`npm run dev`) compiles pages on first visit. To try the optimised build, as the browser tests and load test do, use:

```
npm run build
npm start                  # http://localhost:3000
```

`npm start` runs with `NODE_ENV=production`, but it keeps the development stand-ins as long as `APP_ENV=development`. Only `APP_ENV=staging` or `production` requires real providers and HTTPS (§4.1).

## 3.7 Running everything in containers

```
docker compose --profile app up -d --build
```

This builds the application image, runs migrations at start-up, and starts the app on port 3000 alongside the retention worker. The containers read `.env` but are pointed at the compose PostgreSQL and Redis services.

## 3.8 Updating to a new version

1. Unzip the new version into a fresh folder.
2. Copy your existing `.env` into it. The keys must stay the same, or stored numbers can't be decrypted.
3. Run `npm ci` and then `npm run db:migrate`. Migrations are additive and safe to re-run.
4. Restart `npm run dev` (or `npm start`) and `npm run jobs`.

## 3.9 Resetting the local environment

```
docker compose down -v      # deletes the local database and Redis data
docker compose up -d --wait
npm run db:migrate && npm run seed
```

> **Keep your keys**
>
> `docker compose down -v` deletes all local data, which is fine for development. Never regenerate keys with `-force` on a system with real users: every encrypted number and chat would become unreadable.

CHAPTER 4

# Configuration reference

All configuration comes from environment variables, normally kept in `.env`. The application validates them at start-up and refuses to run with missing or malformed values, naming the setting at fault. `.env.example` lists every setting with its development default.

## 4.1 Environment modes

**Table 4.1:** What `APP_ENV` allows

| Behaviour | development | staging / production |
|---|---|---|
| Development CAPTCHA checkbox | allowed | refused at start-up |
| Emails captured to the outbox | allowed | refused at start-up |
| Resend test sender (`@resend.dev`) | allowed | refused at start-up |
| `/dev/outbox` page and API | available | return *not found* |
| Plain `http://` `APP_URL` | allowed | refused (HTTPS required) |
| Seed script | runs | refuses |
| HSTS and `upgrade-insecure-requests` | only when `APP_URL` is HTTPS | sent |

`NODE_ENV` is managed by Next.js (`npm run dev` sets `development`, `npm start` sets `production`). It only affects build optimisation and hot reload; `APP_ENV` decides what's safe. If `APP_ENV` is unset, it follows `NODE_ENV`.

| Variable | Default | Meaning |
|---|---|---|

## 4.2 Core settings

**Table 4.2:** Core settings

| Variable | Default | Meaning |
|---|---|---|
| `APP_ENV` | `production` | See Table 4.1. `.env.example` is set up for the live server; `./start.sh setup` switches a laptop's copy to `development`. |
| `APP_URL` | `https://pryingeyes.idealsoftwaresolutions.com` | The server's public address (*Site address*, first in the YOUR SETTINGS block). The base of every link it sends (email confirmations, `/t/<code>`, `/g/<invite>`, QR codes); also used for the same-origin check and WebAuthn. In staging and production it must be `https` and not `localhost`, e.g. `https://pryingeyes.idealsoftwaresolutions.com`. |
| `RP_ID` / `RP_NAME` | empty / PryingEyes | Passkey domain. Empty means `APP_URL`'s host. If set, it must be that host or a parent domain of it; changing it later invalidates existing passkeys. |
| `DATABASE_URL` | empty | Empty: the built-in database (PGlite) in `DATA_DIR`/postgres, opened only by the website. Set: a PostgreSQL server, e.g. `postgresql://user:pass@127.0.0.1:5432/pryingeyes`. |
| `REDIS_URL` | empty | Empty: the in-memory store inside the website (reset on restart). Set: a Redis server, needed to run several copies of the app. |
| `DATA_DIR` | `./var/data` | Where the built-in database keeps its files. Back it up together with `.env`. |
| `BACKGROUND_JOBS` | `on` | `on`: the website runs retention and the missed-call sweep itself. `off`: run `npm run jobs` separately (needs `DATABASE_URL`). |
| `ADMIN_EMAILS` | empty | Comma-separated. A verified account with one of these emails becomes an admin automatically (audited). |
| `DEFAULT_COUNTRY` | `SZ` | Country assumed for numbers typed without `+`. |
| `TRUST_PROXY_HOPS` | `1` | Number of proxies in front of the app. Picks the real client IP from `X-Forwarded-For`; set it to match your proxy chain. |
| `AUDIT_ANCHOR_DIR` | `./var/anchors` | Where daily write-once anchor files go. |

| Variable | Default | Meaning |
|---|---|---|
| LOG_LEVEL / LOG_FILE | info / stdout | Log verbosity and optional file. |

## 4.3 Secrets

Generated by `npm run keys:generate`; see Table 2.3 for what each protects.

| Variable | Format |
|---|---|
| MASTER_KEK, TOKEN_PEPPER, BLIND_INDEX_KEY, SECRETS_HMAC_KEY | 32 random bytes, base64 |
| MASTER_KEK_VERSION | Integer, default 1; increase it when rotating |
| KEK_PREVIOUS | JSON of earlier keys, e.g. `{"1":"<base64>"}` |
| EVIDENCE_SIGNING_KEY | Ed25519 private key, PKCS#8 DER, base64 |
| TURN_SECRET | Random URL-safe string (32+ characters), shared with coturn's `static-auth-secret` |
| KEY_PROVIDER | `local` (a cloud KMS provider is a Phase 2 item) |

## 4.4 Providers

**Table 4.3:** Provider settings

| Variable | Values | Meaning |
|---|---|---|
| CAPTCHA_PROVIDER | `dev` \| `turnstile` | Human check. |
| TURNSTILE_SITE_KEY, TURNSTILE_SECRET | | Cloudflare Turnstile keys. |
| TURN_URLS | comma-separated | TURN server URLs given to browsers, e.g. `turn:turn.example.com:3478?transport=udp`, `turns:turn.example.com:5349?transport=tcp`. Must be public in production. |
| CALL_RELAY_ONLY | `true` \| `false` | Force all call audio through TURN, which hides both people's IP addresses. Must be `true` outside development. |
| CALL_RING_SECONDS | default 45 | How long an in-app call rings before it becomes a missed call. |
| EMAIL_PROVIDER | `outbox` \| `resend` | Verification, one-time-code and notification emails. `resend` is required in staging and production. |
| RESEND_API_KEY | `re_...` | Resend API key with sending access (`https://resend.com/api-keys`). Checked at start-up when `EMAIL_PROVIDER=resend`. |

| Variable | Values | Meaning |
|---|---|---|
| EMAIL_FROM | | Sender, on a domain verified in Resend, e.g. PryingEyes <no-reply@yourdomain.com>. The test sender onboarding@resend.dev works only in development and only delivers to the Resend account owner. |
| EMAIL_REPLY_TO | optional | Reply-to address, e.g. a monitored support inbox. |
| AT_SANDBOX | true \| false | Use the sandbox endpoints. |

## 4.5 Policy defaults

These implement rule 7 of the specification. Each can be changed without code changes.

**Table 4.4:** Gate budgets, lockouts and caps

| Variable | Default | Meaning |
|---|---|---|
| GATE_FLOOR_MS | 200 | Every gate response waits at least this long, so timing can't reveal whether a code exists. |
| RL_SIGNUP_PER_IP_HOUR | 30 | New accounts per network address per hour (raise it where many people share one connection). |
| RL_IP_PER_HOUR | 30 | Gate requests per IP address per hour. |
| RL_CALLER_PER_DAY | 10 | New gate attempts per caller per day (UTC). |
| RL_CALLER_TOKEN_PER_DAY | 3 | Attempts per caller per token per day. |
| RL_TOKEN_PER_DAY | 50 | Attempts per token per day, from everyone. |
| RL_FAILS_BEFORE_LOCK | 5 | Failed checks before a caller is locked out. |
| RL_FAIL_WINDOW_SEC | 900 | Window for counting those failures (15 minutes). |
| RL_LOCK_BASE_SEC / RL_LOCK_MAX_SEC | 3600 / 86400 | First lockout is 1 hour, doubling on each repeat within 7 days, up to 24 hours. |
| RL_TOKEN_FAILS_BEFORE_FREEZE | 10 | Failures against one token before it freezes and the owner is alerted. |
| RL_TOKEN_FAIL_WINDOW_SEC | 3600 | Window for token failures (1 hour). |
| TIER2_DAILY_CAP | 3 | Tier 2 contacts a caller can complete per day, across all tokens. |
| MAX_TOKENS_PER_NUMBER | 10 | Live tokens (active, paused or frozen) per protected number. |

## Fixed limits (in code)

| | |
|---|---|
| Per-caller requests in flight at the gate | 2 |
| Gate session after passing checks | 30 minutes |
| Tier 3 consent window | 24 hours (unanswered counts as declined) |
| Reveal grant | single use, 60 seconds |
| One-time codes | 6 digits, 10 minutes, 5 attempts; 5 sends per hour per subject, 20 per hour per IP |
| Password sign-in | 20 attempts per 15 minutes per IP; 10 per 15 minutes per email |
| Sign-ups | 10 per hour per IP |
| Sessions | 12-hour idle timeout, 30-day maximum, bound to the browser |
| Step-up freshness | 10 min (most owner actions); 15 min (data export, erasure, admin); 30 min (removing a number) |
| Chat | 2,000 characters per message; 30 messages per minute; Tier 3 messages disappear after 24 hours |
| Reports | 20 per reporter per day; up to 20 attached messages |
| Handles | 3–24 characters: letters, numbers, underscore, dot; generated as `adjective_noun_NN` |
| Protected numbers per account | 5 |

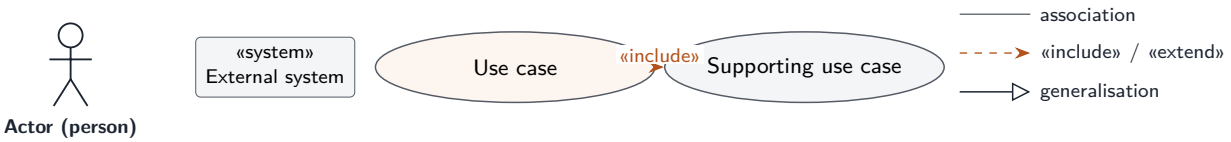
CHAPTER 5

# Use-case model

This chapter is the use-case model of the Phase 1 system. It starts with a context diagram that shows every actor and the subsystems they use. Six detailed use-case diagrams follow, one per subsystem. Chapter 6 specifies each use case in full.

## 5.1 Notation

The diagrams follow UML 2 use-case notation.




- An **«include»** arrow points from a use case to a step it always performs. For example, *Create a token* always includes *Recommend a tier*.
- An **«extend»** arrow points from an optional behaviour to the use case it extends, with the condition in brackets. For example, *Confirm a one-time code* extends *Pass the decode gate* only for Tier 2 and Tier 3 tokens.
- A **generalisation** arrow means the child actor can do everything the parent can. Callers and owners are both *registered users*.
- Grey ellipses are supporting use cases that no actor starts directly.

## 5.2 Actors

**Table 5.1:** Actors

| Actor | Description |
|---|---|
| Visitor | Anyone not signed in. Can open a shared link, read public pages and create an account. |
| Registered user | Any signed-in account. Parent of Caller and Owner; one account can play both roles. |
| Caller | A registered user trying to reach someone through a code. Signs up with a phone number, which only confirms they are a real person. |
| Owner | A registered user who protects numbers and issues tokens. Must have a second factor (passkey or authenticator app) and must have used it in the current session. |
| Moderator | Staff role. Reviews abuse reports; sees report metadata and only the messages a reporter attached, never phone numbers. |
| Administrator | Staff role. Runs break-glass actions; every action needs a second, different administrator to approve it. |
| Operator | Whoever has shell access to the server. Runs migrations, seeds, the admin bootstrap and audit verification. |
| Third-party verifier | Someone outside PryingEyes, such as a platform or police officer, checking an owner's signed evidence pack. |
| Scheduler | The retention worker (`npm run jobs`), running every 10 minutes. |
| Messaging provider | «system» Delivers email (Resend) in production; the development outbox locally. PryingEyes sends no text messages. |
| CAPTCHA service | «system» Cloudflare Turnstile in production; a development checkbox locally. |

## 5.3 System context

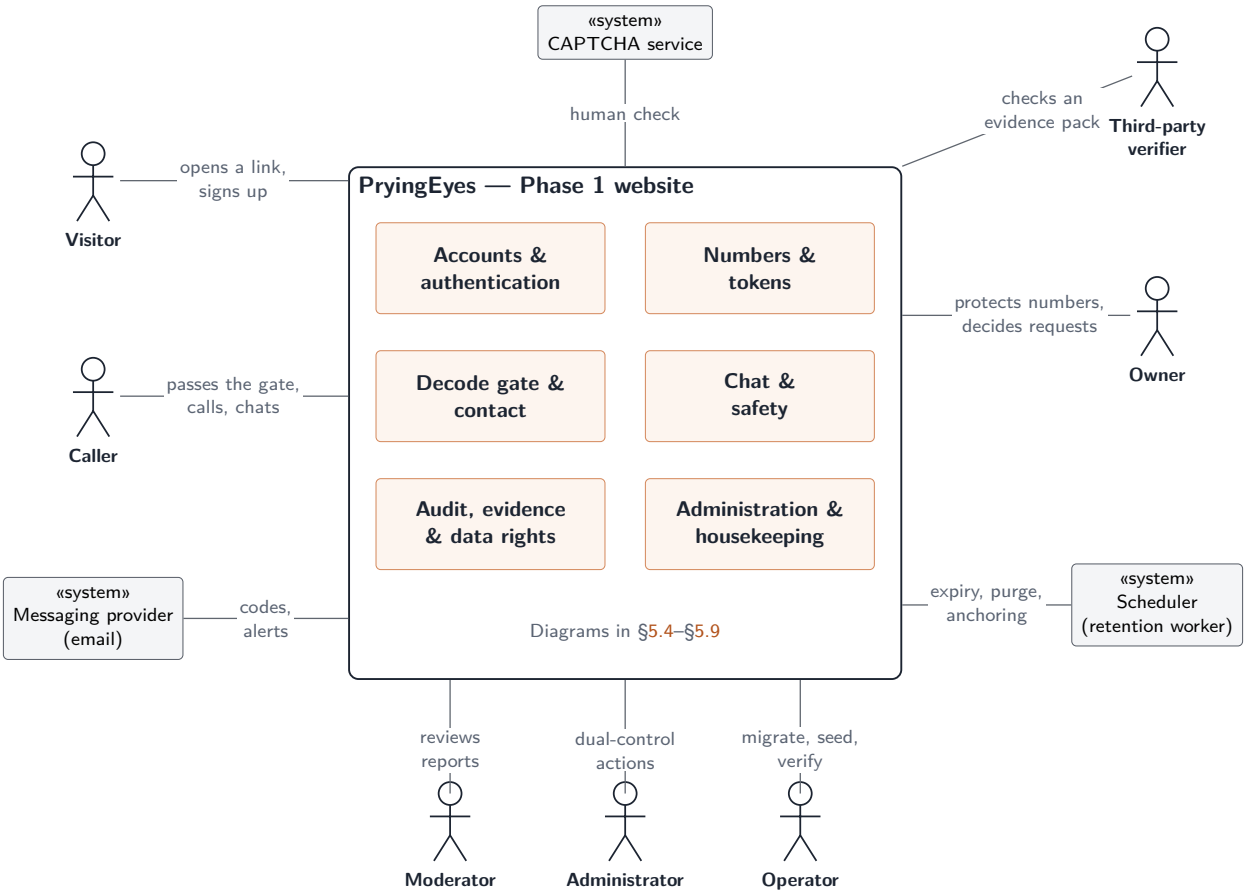



**Figure 5.1:** System context: actors and subsystems

Figure 5.1 shows who interacts with the system. Each subsystem has its own diagram below. Table 6.1 in Chapter 6 lists which actor takes part in which use case.

## 5.4 Accounts and authentication

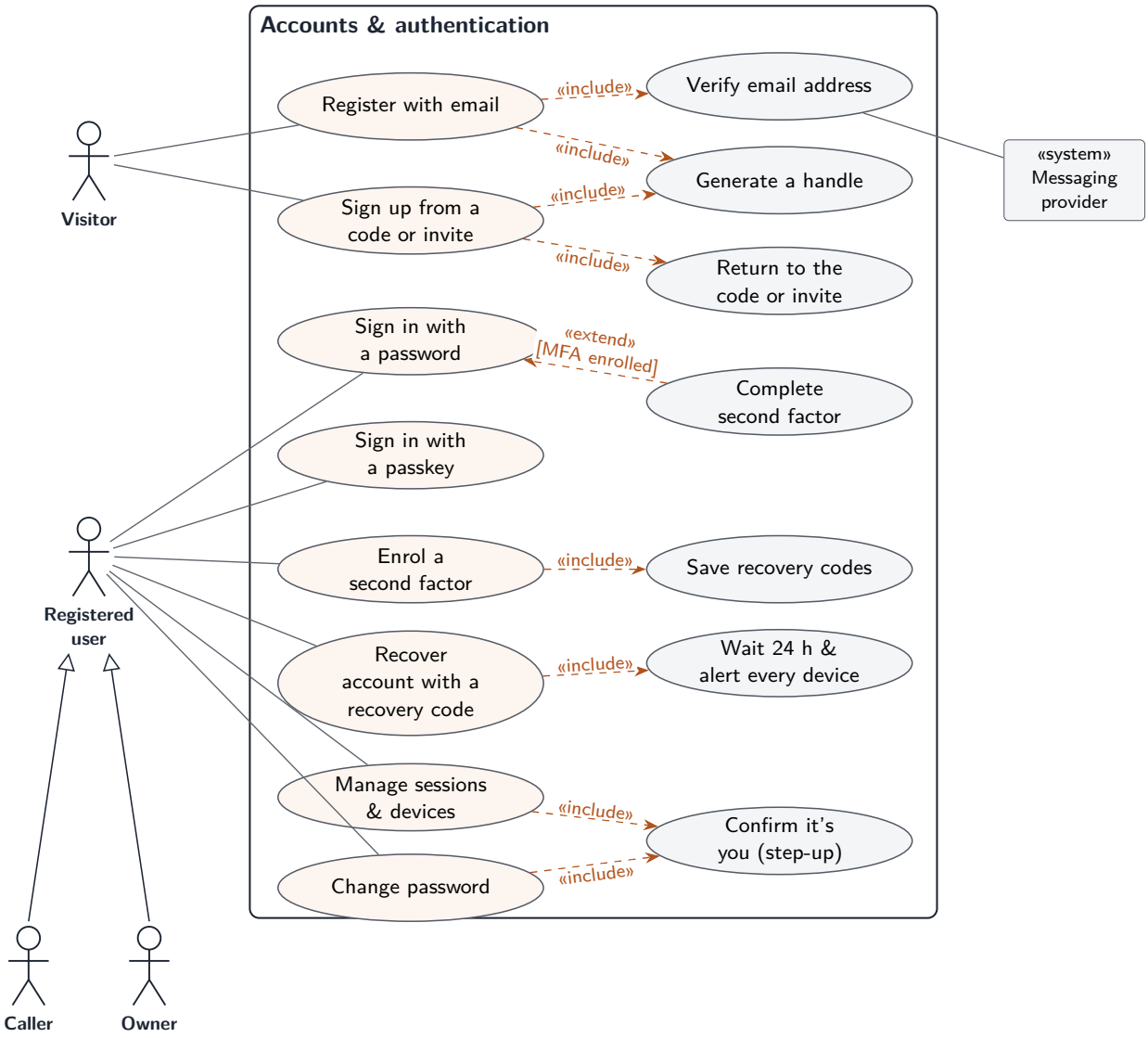



**Figure 5.2:** Use cases — accounts and authentication

People register with an email. Someone who arrives from a contact code or group invite signs up the same way and returns to it afterwards. Sign-up includes *Generate a handle*: a free random handle is suggested and can be kept, replaced or typed over. Second-factor checks extend password sign-in only once a factor is enrolled. Passkey sign-in satisfies both factors at once. Session management and password changes need a recent second-factor confirmation (step-up). Someone who has lost their second factor uses a recovery code instead. That starts a 24-hour cooling-off period, during which every signed-in device is alerted and can cancel.

## 5.5 Numbers and tokens

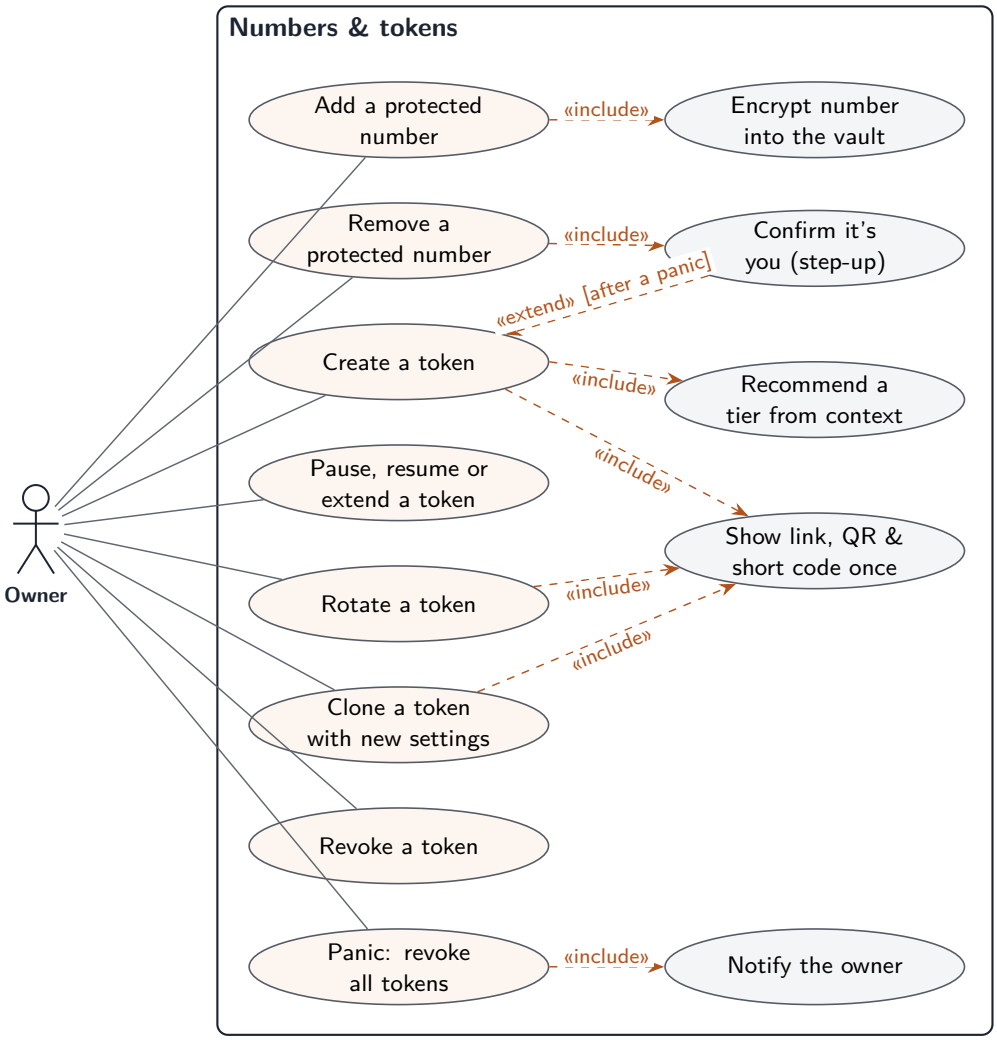



**Figure 5.3:** Use cases — numbers and tokens

Only owners take part in this subsystem. A number is encrypted the moment it's added. A token's link, QR code and short code are shown once, because only a keyed fingerprint is stored. Rotation replaces the codes and kills the old ones in one step. After a panic revoke-all, creating a token requires a fresh second-factor check, and so do rotating, cloning and resuming one (shown on *Create a token* only, for readability). The panic itself never asks for one. Every action here writes an audit event; that's left off the diagram for readability.

## 5.6 Decode gate and contact

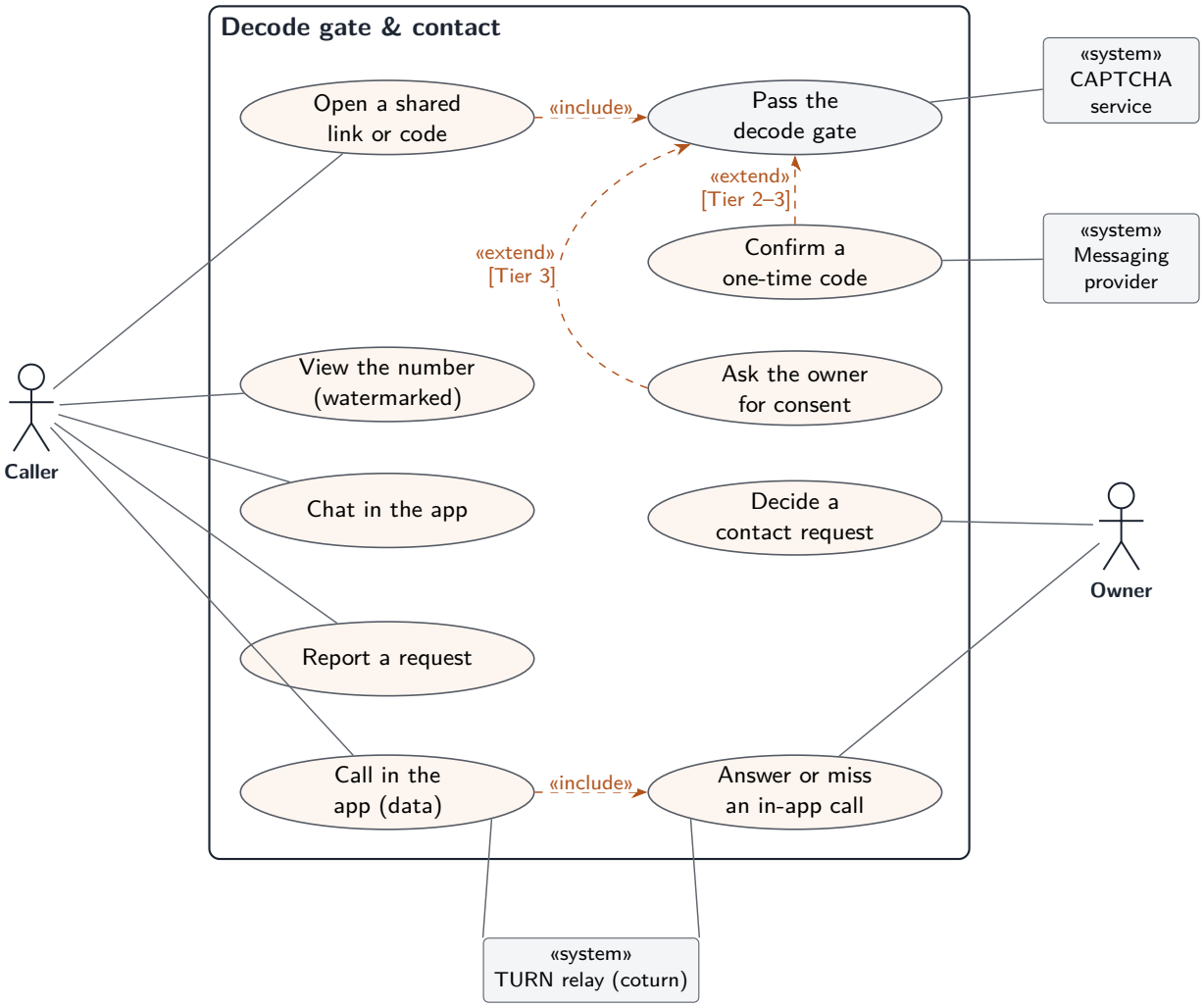



**Figure 5.4:** Use cases — decode gate and contact

This subsystem is what the original brief describes as decoding the number on the system first. Opening a code always includes passing the gate: sign-in, a human check and server-side budgets, in that order. Confirming a one-time code extends the gate for Tier 2 and Tier 3 tokens. Asking the owner extends it for Tier 3. Once through, the caller picks an outcome the tier allows:

- A watermarked reveal, only for codes backed by a phone number (Tiers 1–2, or Tier 3 after *Reveal once*).
- An in-app call over data, through PryingEyes' own TURN relay. No phone numbers are used, and IP addresses stay hidden.
- A chat.

The owner decides Tier 3 requests: approve calls and chat, reveal once, decline, or decline and block. An in-app call includes the owner answering it. If the owner's app is closed, the call becomes a missed call (G11). The messaging provider delivers one-time codes.

## 5.7 Chat and safety

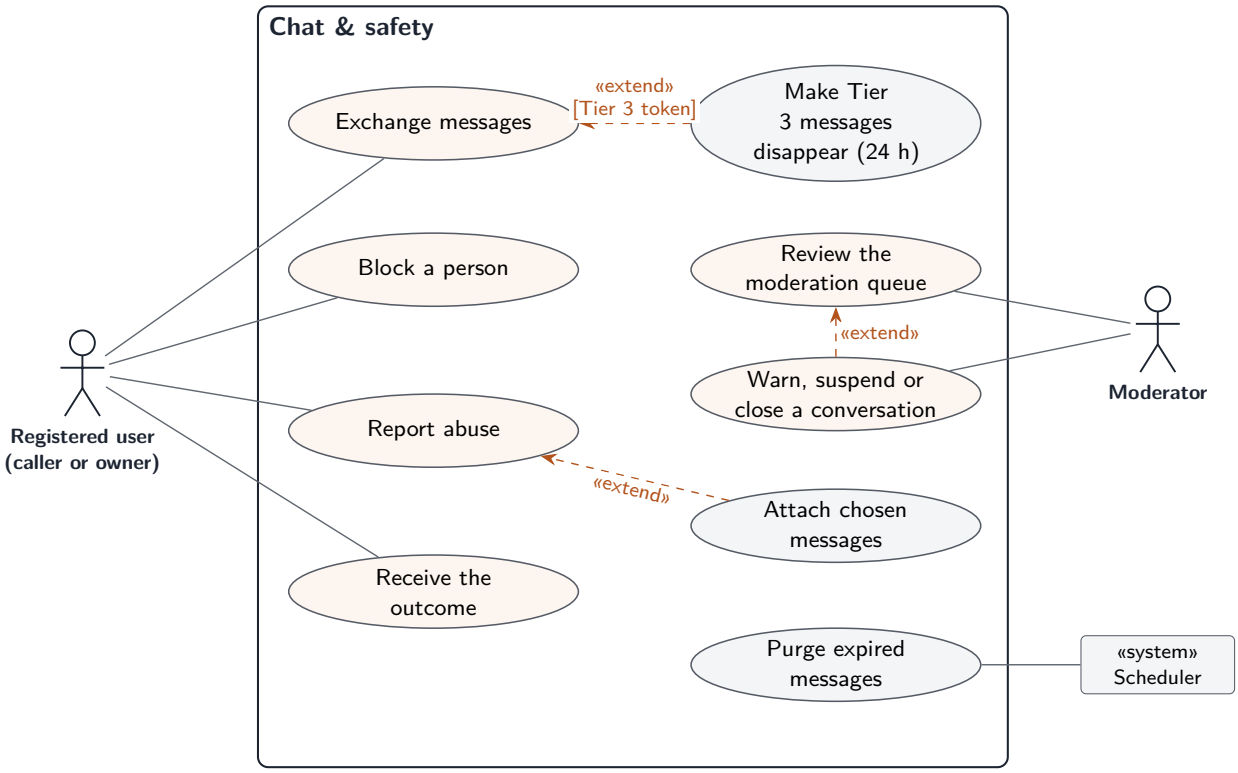



**Figure 5.5:** Use cases — chat and safety

Chats open from the gate, so neither side ever sees the other's number. Blocking works in both directions and also stops the blocked person passing the gate on any of the blocker's codes. A report carries only the messages the reporter chooses to attach. Moderators work a queue with a 60-minute first-response target. They can dismiss, warn, suspend (which signs the person out and pauses their tokens) or close a conversation.

## 5.8 Audit, evidence and data rights

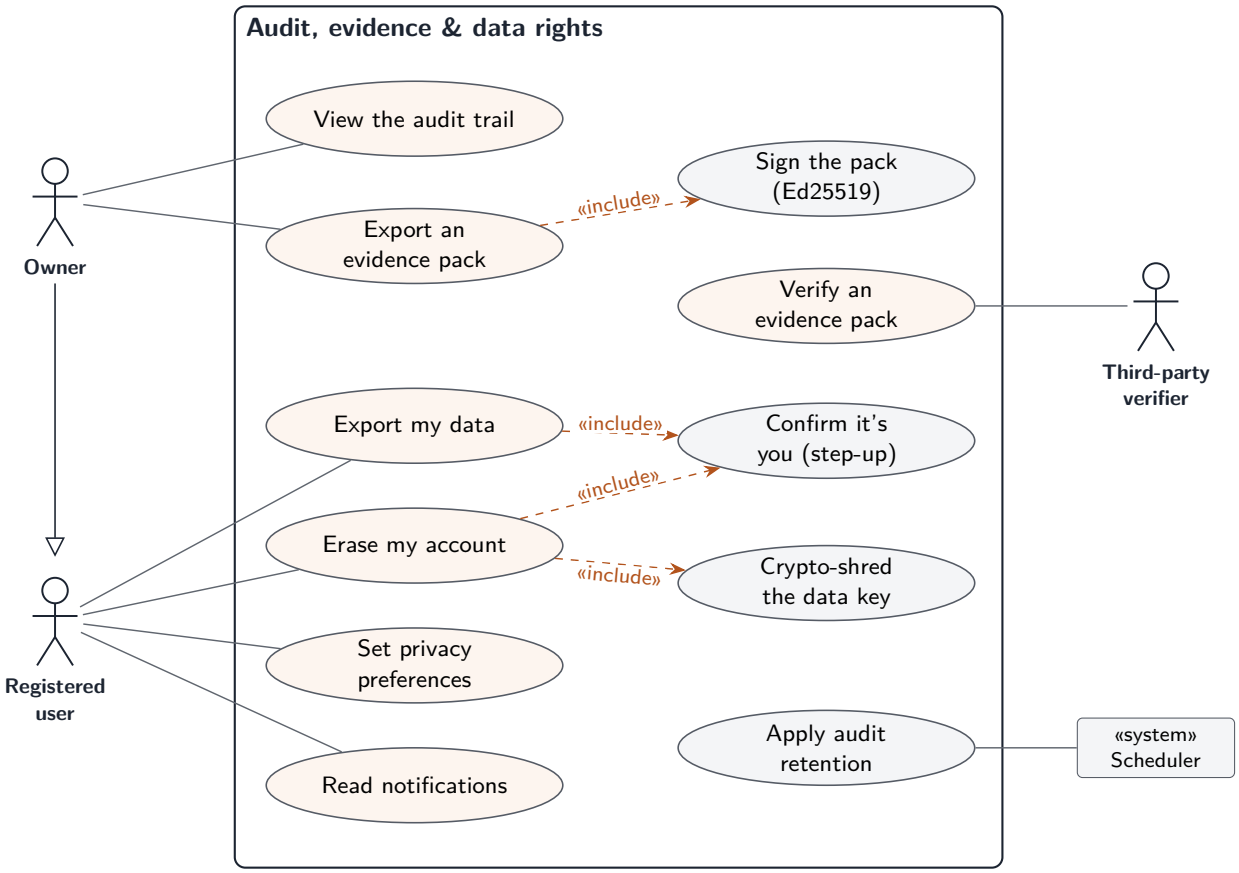



**Figure 5.6:** Use cases — audit, evidence and data rights

Owners see every request against their codes in a tamper-evident audit trail. They can export it as an evidence pack: a JSON file signed with Ed25519 and a printable PDF. Anyone can check a pack's signature at `/api/evidence/verify`. Every registered user can download all their data or erase their account. Erasure deletes the account's rows and its data key, so any copy left in a backup can't be decrypted. The scheduler applies the audit-retention period each owner has chosen.

## 5.9 Administration and housekeeping

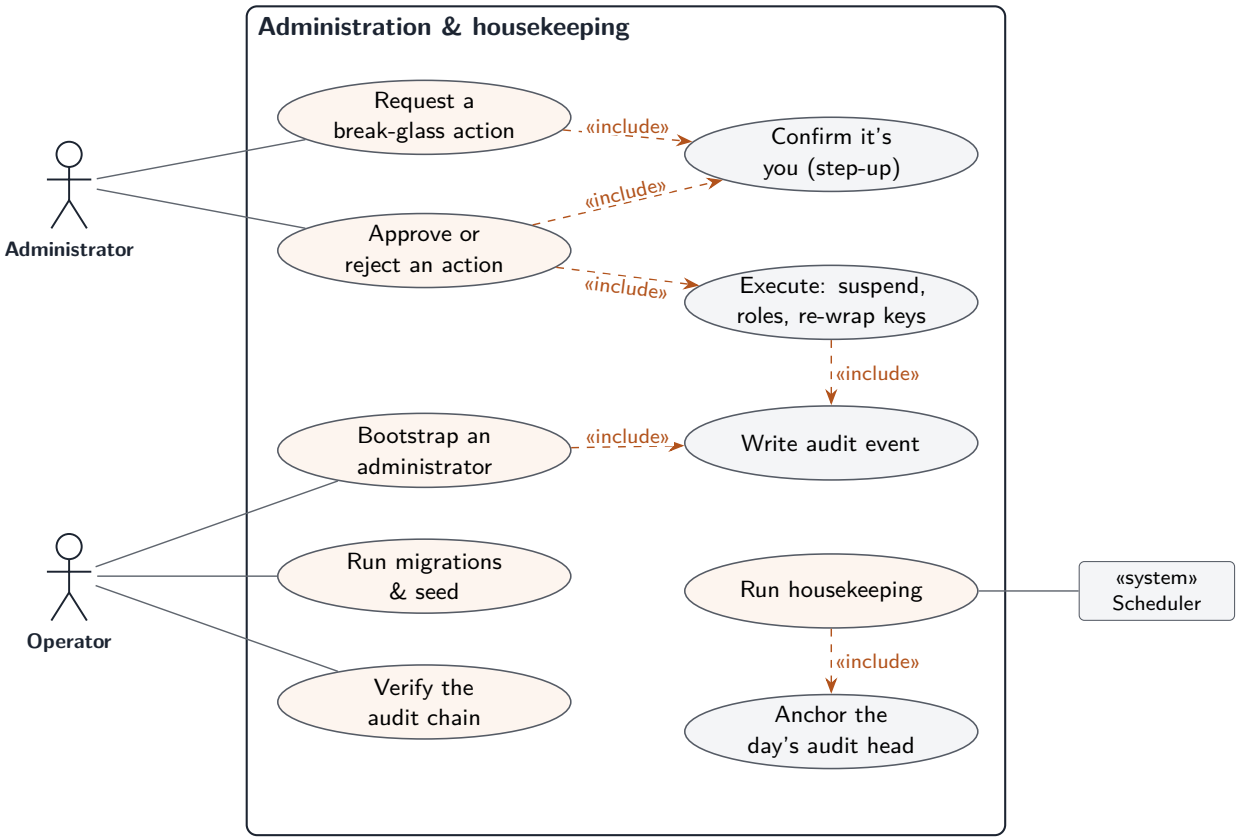



**Figure 5.7:** Use cases — administration and housekeeping

Break-glass actions are suspend or unsuspend a user, grant or revoke a role, and re-wrap every data key after a key rotation. One administrator requests the action with a reason, and a *different* administrator approves or rejects it within 24 hours. Both steps need a recent second-factor check and are written to the audit chain. Operators bootstrap the first administrators from the server shell, which is also audited. The scheduler expires tokens and consent requests, purges data that has passed its retention period, and anchors each completed day's audit head to a write-once file.

CHAPTER 6

# Use-case specifications

Each use case below lists its actors, preconditions, main flow, alternative and error flows, and the result. IDs match the diagrams in Chapter 5: **A** accounts, **T** tokens, **G** gate, **C** chat and safety, **D** data and evidence, **O** operations. Unless stated otherwise, every use case writes at least one event to the audit chain.

## 6.1 Actor and use-case matrix

**Table 6.1:** Which actor takes part in which use case (● primary, ○ secondary)

| ID | Use case | *Visitor* | *Reg. user* | *Caller* | *Owner* | *Moderator* | *Admin* | *Operator* | *Verifier* | *Scheduler* | *Messaging* |
|---|---|---|---|---|---|---|---|---|---|---|---|
| A1 | Register with email | ● | | | | | | | | | ○ |
| A2 | Sign up from a code or invite | ● | | ● | | | | | | | ○ |
| A3 | Verify email address | ● | | | | | | | | | ○ |
| A4 | Sign in with a password | | ● | | | | | | | | |
| A5 | Sign in with a passkey | | ● | | | | | | | | |
| A6 | Enrol a second factor | | ● | | ● | | | | | | |
| A7 | Confirm it's you (step-up) | | ● | | | | | | | | |
| A8 | Recover account with a recovery code | | ● | | | | | | | | ○ |
| A9 | Manage sessions and devices | | ● | | | | | | | | |
| A10 | Change password | | ● | | | | | | | | |
| A11 | Generate a handle | ● | | | | | | | | | |
| T1 | Add a protected number | | | | ● | | | | | | |
| T2 | Remove a protected number | | | | ● | | | | | | |
| T3 | Create a token | | | | ● | | | | | | |
| T4 | Pause, resume or extend a token | | | | ● | | | | | | |
| T5 | Rotate a token | | | | ● | | | | | | |
| T6 | Clone a token | | | | ● | | | | | | |
| T7 | Revoke a token | | | | ● | | | | | | |
| T8 | Panic: revoke all tokens | | | | ● | | | | | | ○ |
| G1 | Open a shared link or code | ● | | ● | | | | | | | |
| G2 | Pass the decode gate | | | ● | ○ | | | | | | |
| G3 | Confirm a one-time code | | | ● | | | | | | | ○ |
| G4 | Ask the owner for consent | | | ● | ○ | | | | | | ○ |

| ID | Use case | Visitor | Reg. user | Caller | Owner | Moderator | Admin | Operator | Verifier | Scheduler | Messaging |
|---|---|---|---|---|---|---|---|---|---|---|---|
| G5 | Decide a contact request | | | ○ | ● | | | | | | |
| G6 | View the number (watermarked) | | | ● | ○ | | | | | | |
| G8 | Chat in the app | | | ● | ● | | | | | | |
| G9 | Report a request | | | ● | ● | ○ | | | | | |
| C1 | Exchange messages | | ● | | | | | | | ○ | |
| C2 | Block a person | | ● | | | | | | | | |
| C3 | Report abuse | | ● | | | ○ | | | | | |
| C4 | Review the moderation queue | | | | | ● | | | | | |
| C5 | Take a moderation action | | ○ | | | ● | | | | | ○ |
| D1 | View the audit trail | | | | ● | | | | | | |
| D2 | Export an evidence pack | | | | ● | | | | | | |
| D3 | Verify an evidence pack | | | | | | | | ● | | |
| D4 | Export my data | | ● | | | | | | | | |
| D5 | Erase my account | | ● | | | | | | | | |
| D6 | Set privacy preferences | | ● | | | | | | | | |
| D7 | Read notifications | | ● | | | | | | | | ○ |
| O1 | Request a break-glass action | | | | | | ● | | | | |
| O2 | Approve or reject an action | | | | | | ● | | | | |
| O3 | Bootstrap an administrator | | | | | | | ● | | | |
| O4 | Run migrations and seed | | | | | | | ● | | | |
| O5 | Verify the audit chain | | | | | | | ● | | | |
| O6 | Run housekeeping | | | | | | | | | ● | |
| O7 | Rotate the key-encryption key | | | | | | ● | ● | | | |

## 6.2 Accounts and authentication

### A1 Register with email

**Actors** Visitor; Messaging provider (email).

**Precondition** Not signed in.

**Main flow**

1. The visitor opens **Create account** → **Protect my number**.
2. The handle field is already filled with a free generated handle (A11).
3. The visitor enters an email and a password of at least 10 characters, then the handle (already filled in), and confirms they are 18 or older.
4. The system hashes the password with Argon2id, creates a *pending* account and emails a 24-hour confirmation link (A3).
5. The page says “Check your email”.

**Alternatives**
- Email already registered: the page shows the same "Check your email" message, and the real account holder gets an email saying someone tried to sign up. This prevents account enumeration.
- Typed handle taken: error `handle_taken` with a fresh `suggestion`, which the form drops in.
- Typed handle invalid (an `@`, a hyphen, too short or too long): the box cleans it as it is typed and refuses anything with `@`, which is usually browser autofill. The server also answers `invalid_handle` with a corrected `suggestion`.
- Weak password, no 18+ confirmation, or too many sign-ups per hour from one network address (`RL_SIGNUP_PER_IP_HOUR`, default 30): refused with a message.

**Result**
A pending account that can't sign in until the email is confirmed.

## A2 Sign up from a code or invite

**Actors**
Visitor (soon a Caller or group member); Messaging provider (email).

**Precondition**
The person opened a contact code (`/t/<code>`) or a group invite (`/g/<invite>`) while signed out.

**Main flow**
1. The gate or join screen offers **Create an account** and **Sign in**. Both carry the code or invite as `next`.
2. They sign up with email and password (A1). The confirmation email's link carries `next` too; the server accepts only a local path.
3. They confirm the email (A3), sign in, and land back on the same code or invite, already filled in. In the phone app they return to the screen they started from.

**Alternatives**
- Already have an account: **Sign in** returns them the same way, including after the second factor.
- A `next` that points to another site is ignored, and they land on **My codes**.

**Result**
A signed-in account back on the code or invite. People see the handle, never the email.

## A3 Verify email address

**Actors**
Visitor (from the emailed link).

**Main flow**
1. The visitor opens `/verify-email?token=...`.
2. The system checks the stored hash of the single-use link (valid for 24 hours).
3. The account becomes *active*.

**Alternatives**
Expired or already-used link: "This link has expired or was already used."

**Result**
The account can sign in.

## A4 Sign in with a password

**Actors**
Registered user.

**Main flow**
1. The user enters an email and password.
2. The system verifies the Argon2id hash and creates a session cookie: random value, SHA-256 stored, HttpOnly, SameSite=Lax.
3. If the user has a second factor, the session is *MFA-pending* and the user is sent to `/login/mfa`. They complete it with a passkey or a 6-digit authenticator code. Codes can't be reused.
4. The user lands on the page they asked for, or the dashboard.

| | |
|---|---|
| **Alternatives** | • Wrong password or unknown email: the same generic error, with equalised timing.<br>• Unverified email: `verification_required`. Suspended account: `account_suspended`.<br>• Throttling: 20 attempts per 15 minutes per IP, 10 per email.<br>• Lost second factor: A8. |
| **Result** | A full session. Owner features also need the session to have used the second factor. |

### A5 Sign in with a passkey

| | |
|---|---|
| **Actors** | Registered user (with a passkey). |
| **Main flow** | 1. The user chooses **Sign in with a passkey**.<br>2. The browser asks for the device's screen lock (fingerprint, face or PIN).<br>3. The system verifies the WebAuthn assertion (user verification required) and the signature counter.<br>4. A full session starts with MFA satisfied. |
| **Alternatives** | Challenge older than 5 minutes, unknown credential or failed verification: `passkey_failed` or `challenge_expired`. |
| **Result** | A full session. A passkey counts as both factors. |

### A6 Enrol a second factor

| | |
|---|---|
| **Actors** | Registered user (required for owners). |
| **Main flow** | 1. In onboarding or **Settings**, the user chooses **Create a passkey** (recommended) or **Use an authenticator app**.<br>2. **Passkey:** the browser creates a credential and the server verifies and stores its public key.<br>3. **Authenticator app:** the server shows a QR code and the key. The user confirms with a current code, and the secret is stored encrypted.<br>4. On the first factor, the system shows 10 one-time recovery codes (`XXXXX-XXXXX`) to download, print or copy. |
| **Alternatives** | Adding a further factor needs a step-up (A7). Removing the last factor is refused (`last_factor`). |
| **Result** | The account can become an owner. The current session counts as MFA-verified. |

### A7 Confirm it's you (step-up)

| | |
|---|---|
| **Actors** | Registered user. |
| **Trigger** | The server answers `step_up_required`. This happens for sensitive actions when the last second-factor check is older than the window (Table 4.4, fixed limits), and after a panic revoke-all. |
| **Main flow** | 1. The **Confirm it's you** dialog opens automatically.<br>2. The user uses a passkey or enters an authenticator code.<br>3. The original request is retried automatically. |
| **Result** | The session's `mfa_at` is refreshed. |

### A8 Recover account with a recovery code

| | |
|---|---|
| **Actors** | Registered user who has lost their second factor; Messaging provider (alerts). |
| **Precondition** | Signed in with a password, with the session waiting for its second factor. |

| | |
|---|---|
| **Main flow** | 1. On `/login/mfa`, the user chooses **Use a recovery code** and enters one.<br>2. The code is used up and recovery starts with a **24-hour cooldown**. Every device and the email address are alerted.<br>3. After 24 hours, the user chooses **Finish recovery**. The system removes the passkeys, authenticator secret and remaining codes.<br>4. The user enrols a new factor (A6). |
| **Alternatives** | The real owner cancels from **Settings** during the cooldown. Finishing early is refused with `recovery_cooldown`. There is no SMS recovery (PryingEyes sends none), which defeats SIM-swap takeovers. |
| **Result** | The second factor is reset; owner features are locked until a new one is enrolled. |

### A9 Manage sessions and devices

| | |
|---|---|
| **Actors** | Registered user. |
| **Main flow** | 1. **Settings** → **Signed-in devices** lists each session with its device label and last activity.<br>2. The user signs out any other device. |
| **Rules** | Sessions end after 12 hours idle or 30 days at most. A session cookie used from a different browser is rejected. |

### A10 Change password

| | |
|---|---|
| **Actors** | Registered user. |
| **Main flow** | The user enters the current password and a new one. Without the current password, a step-up (A7) is accepted instead. All other devices are signed out. |

### A11 Generate a handle

| | |
|---|---|
| **Actors** | Visitor (during A1). |
| **Main flow** | 1. The form calls `/api/auth/handle`, which returns a free handle such as `quiet_heron_42` (adjective, noun, number).<br>2. The person keeps it, taps **Suggest another**, or types their own. Typed handles are checked live with `/api/auth/handle?check=...`.<br>3. On submit, an empty field means the server generates one. If a concurrent sign-up takes a generated handle first, the server retries with a new one. |
| **Rules** | Generated handles are never derived from the email, name or phone, because everyone the person contacts sees their handle. |

## 6.3 Numbers and tokens

All use cases in this section need an owner: a verified account, 18+ confirmation, a second factor, and a session that used it.

### T1 Add a protected number

| | |
|---|---|
| **Actors** | Owner. |

**Main flow**
1. The owner picks the country, types the number without the dialling code, and adds an optional private name. The form checks the number against the country as it's typed.
2. The system normalises it to E.164 and computes the blind index.
3. The number is encrypted with the owner's data key, bound to the row id. Only the last two digits are kept readable, as a hint.

**Alternatives**
Same number already added, in any format: `number_exists`. More than 5 numbers: `number_cap`. Invalid number: `invalid_phone`, naming the selected country (for example "That isn't a valid phone number for Eswatini (+268)"). Unknown country: `invalid_country`.

## T2 Remove a protected number

**Actors**
Owner.

**Main flow**
After confirming, the number and every token for it are deleted. Needs a second-factor check within the last 30 minutes.

## T3 Create a token

**Actors**
Owner.

**Main flow**
1. The owner chooses where they will post the code. The system recommends a tier and class (Table 1.1).
2. The owner adjusts the tier (1–3; Tier 4 is disabled), class, expiry, label (up to 40 characters, shown to callers) and number.
3. The system draws a 16-character link code and a 10-character short code from a cryptographic random generator. It stores only their HMAC fingerprints and a 4-character display prefix.
4. The owner sees the link, short code and QR code **once**, with copy and download buttons.

**Alternatives**
10 live tokens on the number: `token_cap`. After a panic, a step-up is needed first.

**Result**
An active token.

## T4 Pause, resume or extend a token

**Actors**
Owner.

**Main flow**
From the token's `...` menu:

- **Pause** stops the token without deleting it.
- **Resume** restarts a paused or frozen token.
- **Extend 48 h** lengthens an expiring token; an expired token becomes active again.

**Rules**
Resuming after a panic needs a step-up.

## T5 Rotate a token

**Actors**
Owner.

**Main flow**
In one transaction, the system issues new codes with the same settings and revokes the old token. The new codes are shown once. Use this when a code is lost, leaked or being targeted.

### T6 Clone a token

**Actors** Owner.

**Main flow** Creates a new token for the same number from an existing one, with any settings changed (via the API: tier, label, class, expiry).

### T7 Revoke a token

**Actors** Owner.

**Main flow** After confirming, the token is revoked. It takes effect immediately, including for callers already part-way through the gate and for unused reveal grants.

### T8 Panic: revoke all tokens

**Actors** Owner; Messaging provider (alert email).

**Main flow**

1. The owner chooses **Panic: revoke all** and confirms.
2. In one transaction, every live token is revoked and every open request is declined. The account is marked with the panic time.
3. The owner is notified. The response reports how many tokens were revoked and how long it took (under one second).

**Rules** No step-up is needed, because revoking is always safe. Creating, rotating, cloning or resuming tokens afterwards needs a second-factor check made after the panic.

## 6.4 Decode gate and contact

### G1 Open a shared link or code

**Actors** Visitor or Caller.

**Main flow**

1. The person opens `/t/<code>`, or types a link or short code on the home page. Spaces, dashes, lower case and confusable letters (O/0, I/L/1) are accepted.
2. Signed out: they create an account or sign in, and come back to the same code (A2).
3. Signed in: they continue to G2.

### G2 Pass the decode gate

**Actors** Caller; CAPTCHA service; Owner (sees the result in their feed).

**Main flow**

1. The caller ticks the human check and chooses **Continue**.
2. The server checks, in order:
   - the caller's lockout and in-flight limit;
   - the IP budget and the caller's daily budget;
   - the CAPTCHA;
   - the token's status;
   - whether the owner has blocked the caller;
   - the caller-per-token and per-token budgets.
3. On success, a gate request is created (valid for 30 minutes) and the caller's region is recorded. The caller sees the token's label and tier.

**Alternatives**

- Unknown, paused, frozen, revoked, expired or consumed code, a block, or a per-token budget used up: all return the same `unavailable` response, after the same 200 ms floor.
- Budget or lockout: `limited`, with the wait time.
- CAPTCHA failure: `captcha_failed`. This is checked before the token lookup, so it reveals nothing about the code.
- Each failure counts toward the caller's lockout (5 in 15 minutes) and the token's freeze (10 in 1 hour). A freeze alerts the owner in-app and by email.
- Reloading the page resumes the open request without using more budget.

## G3 Confirm a one-time code (Tier 2 and 3)

**Actors** Caller; Messaging provider.

**Main flow** The caller chooses **Send me a code**. A 6-digit code goes to their verified email. They enter it and the request is marked *verified* ("account + human check + code").

**Alternatives** Wrong code: `otp_wrong`, which counts as a failure toward lockout and freeze. The caller can request a new code.

## G4 Ask the owner for consent (Tier 3)

**Actors** Caller; Owner (notified); Messaging provider (email).

**Main flow**

1. After G3, the caller may add a note of up to 280 characters, stored encrypted under the owner's key.
2. The caller chooses **Send request**.
3. The owner is notified in-app and by email.
4. The caller's page shows **Waiting for the owner** and checks every 5 seconds.

**Result** A pending request that expires after 24 hours and then counts as declined.

## G5 Decide a contact request

**Actors** Owner; Caller (sees the result).

**Main flow**

1. **Requests** shows each pending request with:
   - the caller's handle;
   - how they were verified;
   - the age of their account and their region;
   - reports against them and earlier requests to this owner;
   - their note.

   It never shows their number.
2. The owner chooses one of:
   - **Allow messages and calls** (the default) — chat and in-app calls;
   - **Show my number once** (only for codes with a number) — one reveal, then chat and calls;
   - **Decline**;
   - **Decline & block** — also blocks the caller from every one of the owner's codes.

**Alternatives** Already decided: `already_decided`. Past 24 hours: `expired`. Someone else's request: `not_found`.

**Result** An approved caller sees their options. A declined caller sees the same *unavailable* page as for an unknown code.

## G6 View the number (watermarked)

**Actors**
Caller; Owner (the view is logged).

**Precondition**
A Tier 1 or 2 request that passed the gate, or a Tier 3 request approved with *Reveal once*.

**Main flow**
1. The caller chooses **Show the number**.
2. The server issues a random, single-use grant valid for 60 seconds.
3. The browser fetches it. The server re-checks the token, decrypts the number and draws it into a PNG, with the caller's handle, the time and the token label repeated diagonally. The response has `Cache-Control: no-store`.
4. The image is shown from a `blob:` address and removed after 60 seconds.

**Rules**
The number never appears as text in the page or in an API response. Each caller gets one reveal per request. The first delivered outcome counts once against the token's budget; a disposable token is then consumed. Tier 2 contacts count toward the caller's daily cap of 3.

## G8 Chat in the app

**Actors**
Caller, Owner.

**Main flow**
1. On the gate, each option is a card that says what will happen: chat (no numbers shared; disappears after 24 hours on Tier 3), an in-app call over data, or view the number (60 seconds, watermarked; only for codes with a number).
2. The caller chooses **Chat in the app**. The system opens, or reuses, the conversation between this caller and this token.
3. On a new conversation the owner is notified in the app and by email ("*@handle* started a chat about *label*"), and the **Chats** menu item shows an unread count. Both parties continue in C1.

## G10 Call in the app (over data)

**Actors**
Caller; Owner; TURN relay (coturn).

**Precondition**
The caller has passed the gate, and the token allows calls (Tier 1–2, or an approved Tier 3 request). This works from the gate or from an open chat while the gate request is valid. No phone numbers are needed.

**Main flow**
1. The caller chooses **Call in the app**. The system creates a *ringing* call and opens the call screen. The browser asks for the microphone while the call rings.
2. The call screen says whether the owner has PryingEyes open, and counts down the ring (`CALL_RING_SECONDS`, default 45).
3. The owner answers (G11). Each side fetches short-lived TURN credentials. The caller's browser sends an offer, the owner's browser answers, and ICE candidates are exchanged through the server.
4. Audio flows browser to browser through the TURN relay, encrypted with DTLS-SRTP. The screens show **Connected**, a timer, **Mute** and **Hang up**.
5. Either side hangs up. The system records the duration and the outcome.

**Alternatives**

- The owner's app is closed or nobody answers: the call becomes *missed* (G11).
- The owner is already on a call: *busy*, and the owner gets a missed-call notice.
- The caller cancels while it rings: *cancelled*; a missed call if the owner wasn't looking.
- The caller already has a call open: `already_in_call`. More than 10 calls in an hour: `rate_limited`.
- Microphone blocked: the screen explains how to allow it, and the call is cancelled.
- No audio within 20 seconds (relay unreachable, very poor network): "We couldn't connect the call", and the call ends.
- A network change mid-call: one ICE restart is tried; if that fails, the call ends.
- Either side stops polling for 30 seconds (closed tab, lost signal): the server ends the call.

**Rules**

Only callers start calls. Relay-only: the server hands out `iceTransportPolicy: relay` and strips host and server-reflexive candidates, so neither person learns the other's IP address. Calls are never recorded. Signalling lives in Redis for at most 10 minutes and is deleted on read; the database keeps metadata only. Calls last at most 2 hours.

### G11 Answer or miss an in-app call

**Actors**

Owner; Scheduler (expires rings).

**Main flow**

1. While PryingEyes is open on any page, the app checks for incoming calls every 3 seconds. A ringing call shows a banner ("*@handle* is calling about *label*"), a ring tone and a flashing tab title.
2. The owner chooses **Answer**, which opens the call screen and connects (G10), or **Decline**.

**Alternatives**

The app is closed, so nothing can ring in a browser. When the ring time runs out, the call becomes *missed*. The owner gets an in-app and email notification ("Missed call from *@handle* about *label*") that links to a chat with the caller, opened quietly if it didn't exist. The chat timeline shows the missed call, so the owner can reply in writing.

**Rules**

The `jobs` worker checks for expired rings every 10 seconds, so missed calls are reported even when nobody has the app open. Native apps will ring through a VoIP push at the same point (section 7.4).

### G9 Report a request

**Actors**

Caller or Owner; Moderator.

**Main flow**

From a gate page (**Report**) or a request, the person picks a reason and adds details. The report joins the moderation queue (C4).

## 6.5 Chat and safety

### C1 Exchange messages

**Actors**

Registered user (owner or caller); Scheduler (purges expired messages).

**Main flow**

Each side writes messages of up to 2,000 characters; the page refreshes every 3 seconds. Messages are encrypted at rest under a key derived from the owner's data key. Read receipts show unless the reader has turned them off.

| | |
|---|---|
| Rules | On Tier 3 tokens, messages disappear after 24 hours. There are no attachments, and the server does not scan content. At most 30 messages per minute per sender. After a token is revoked, the caller can no longer send. The other party is told about new messages at most once every 15 minutes per conversation, and notifications never contain message text. |

### C2 Block a person

| | |
|---|---|
| Actors | Registered user. |
| Main flow | **Block** in a conversation stops messages in both directions. The blocked person also gets *unavailable* when they try any of the blocker's codes. |

### C3 Report abuse

| | |
|---|---|
| Actors | Registered user; Moderator. |
| Main flow | 1. **Report** opens a form with a reason (harassment, threats, scam, spam, impersonation, other) and optional details.<br>2. The reporter ticks the messages to include. Only those are sent.<br>3. The report is stored encrypted and the person is blocked. |
| Rules | At most 20 reports per reporter per day and 20 attached messages per report. |

### C4 Review the moderation queue

| | |
|---|---|
| Actors | Moderator. |
| Main flow | `/moderation` lists open reports, oldest first. Each shows the reason, both handles, how many reports the reported account has received, the reporter's text and attached messages. Reports older than 60 minutes without a response are flagged. |
| Rules | Moderators never see phone numbers or messages that weren't attached. |

### C5 Take a moderation action

| | |
|---|---|
| Actors | Moderator; the reported user and the reporter (both notified). |
| Main flow | The moderator chooses one of:<br>• **Dismiss**;<br>• **Warn** — the reported user gets a warning by email and in-app;<br>• **Close conversation**;<br>• **Suspend account** — signs the user out everywhere and pauses their active tokens.<br>The reporter is told the report was reviewed. |
| Alternatives | Report already handled: `already_handled`. |

## 6.6 Audit, evidence and data rights

### D1 View the audit trail

| | |
|---|---|
| Actors | Owner. |
| Main flow | The dashboard shows the 12 latest events; **Audit trail** pages through everything. Each entry shows the time, the caller's handle, the event, the token label, the region and the outcome. |

### D2 Export an evidence pack

**Actors**
Owner.

**Main flow**
**Download evidence (PDF)** or **Signed evidence (JSON)**, optionally limited with `from` and `to` dates. The JSON contains:

- each event's chain fields (`prev_hash`, `payload_hash`, `hash`);
- the daily anchors;
- the public key;
- an Ed25519 signature over the canonical JSON.

The PDF lists the same events and the signature's fingerprint.

**Rules**
No phone numbers are included.

### D3 Verify an evidence pack

**Actors**
Third-party verifier.

**Main flow**
Post the JSON to `/api/evidence/verify`. The response is `{valid, signedByThisServer}`. Any edit, including removing a single event, makes `valid` false. The public key is published at `/api/evidence/public-key` for offline checks.

### D4 Export my data

**Actors**
Registered user.

**Main flow**
**Download my data** returns a JSON file containing:

- the profile and the user's own numbers (decrypted);
- tokens (settings only, because codes are not stored);
- requests made and received, with notes;
- conversations with messages;
- in-app call metadata, notifications, sessions and activity.

A step-up within 15 minutes is needed if a second factor is enrolled.

### D5 Erase my account

**Actors**
Registered user.

**Main flow**
1. The user types `DELETE MY ACCOUNT` and confirms; a step-up is needed if a second factor is enrolled.
2. The system records the erasure, then deletes the user's rows, which cascade to their keys, numbers, tokens, requests, chats and sessions.
3. Audit payloads that mention the user are nulled; the chain still verifies.
4. Reports keep their moderation value but lose the link to the person.

**Result**
The data is gone from the live system immediately, and from backups within 35 days. Because the data key is deleted, old backup ciphertext is unreadable.

### D6 Set privacy preferences

**Actors**
Registered user.

**Main flow**
Choose how long the audit trail is kept (1, 3, 6, 12, 18 or 24 months; default 12), and whether chats send read receipts.

### D7 Read notifications

| | |
|---|---|
| Actors | Registered user; Messaging provider. |
| Main flow | The handle in the menu shows an unread count. **Settings** lists notifications and marks them read. Notifications also go by email when the address is verified. |

## 6.7 Administration and operations

### O1 Request a break-glass action

| | |
|---|---|
| Actors | Administrator. |
| Main flow | On `/admin`, choose an action (suspend or unsuspend a user, grant or revoke the moderator or admin role, or re-wrap data keys), give a reason of at least 10 characters, and confirm with a step-up (15 minutes). |
| Result | A pending request that expires after 24 hours. |

### O2 Approve or reject an action

| | |
|---|---|
| Actors | A *different* administrator. |
| Main flow | Approve, after a step-up. The action runs and its result is shown; the request, approval and execution are all audited. |
| Alternatives | The same administrator who requested it: `dual_control`. Expired or already decided: refused. |

### O3 Bootstrap an administrator

| | |
|---|---|
| Actors | Operator. |
| Main flow | `npm run admin:create - -handle=<handle> -role=admin\|moderator` on the server grants a role to an existing account and writes an audit event. Use this only for the first two administrators. |

### O4 Run migrations and seed

| | |
|---|---|
| Actors | Operator. |
| Main flow | `npm run db:migrate` applies any new SQL migrations, retrying while the database starts. `npm run seed` creates demo data in development only. |

### O5 Verify the audit chain

| | |
|---|---|
| Actors | Operator (daily, from cron). |
| Main flow | `npm run audit:verify` recomputes every link and checks every anchor file. It exits non-zero and names the first broken row if anything was altered. |

### O6 Run housekeeping

| | |
|---|---|
| Actors | Scheduler (every 10 minutes). |

**Main flow**

One pass:

- expires consent requests and tokens;
- deletes disappearing messages;
- purges old one-time codes, links, grants, sessions, intro notes and outbox entries;
- expires stale admin requests;
- applies each owner's audit-retention period;
- anchors the previous day's audit head.

Retention periods are in Table 11.2.

## O7 Rotate the key-encryption key

**Actors**

Operator and Administrators.

**Main flow**

1. The operator moves the current `MASTER_KEK` into `KEK_PREVIOUS` under its version number, sets a new key, increases `MASTER_KEK_VERSION` and restarts.
2. An administrator requests *re-wrap data keys* (O1) and a second approves it (O2). Every user's data key is re-wrapped under the new KEK; the data itself is not re-encrypted.
3. The operator removes the old key from `KEK_PREVIOUS`.

CHAPTER 7

# Key flows and lifecycles

This chapter shows the runtime behaviour behind the most important use cases: the decode gate's decision flow, how people chat and call privately, a complete Tier 3 contact, and the lifecycles of tokens and gate requests.

## 7.1 Decode gate decision flow

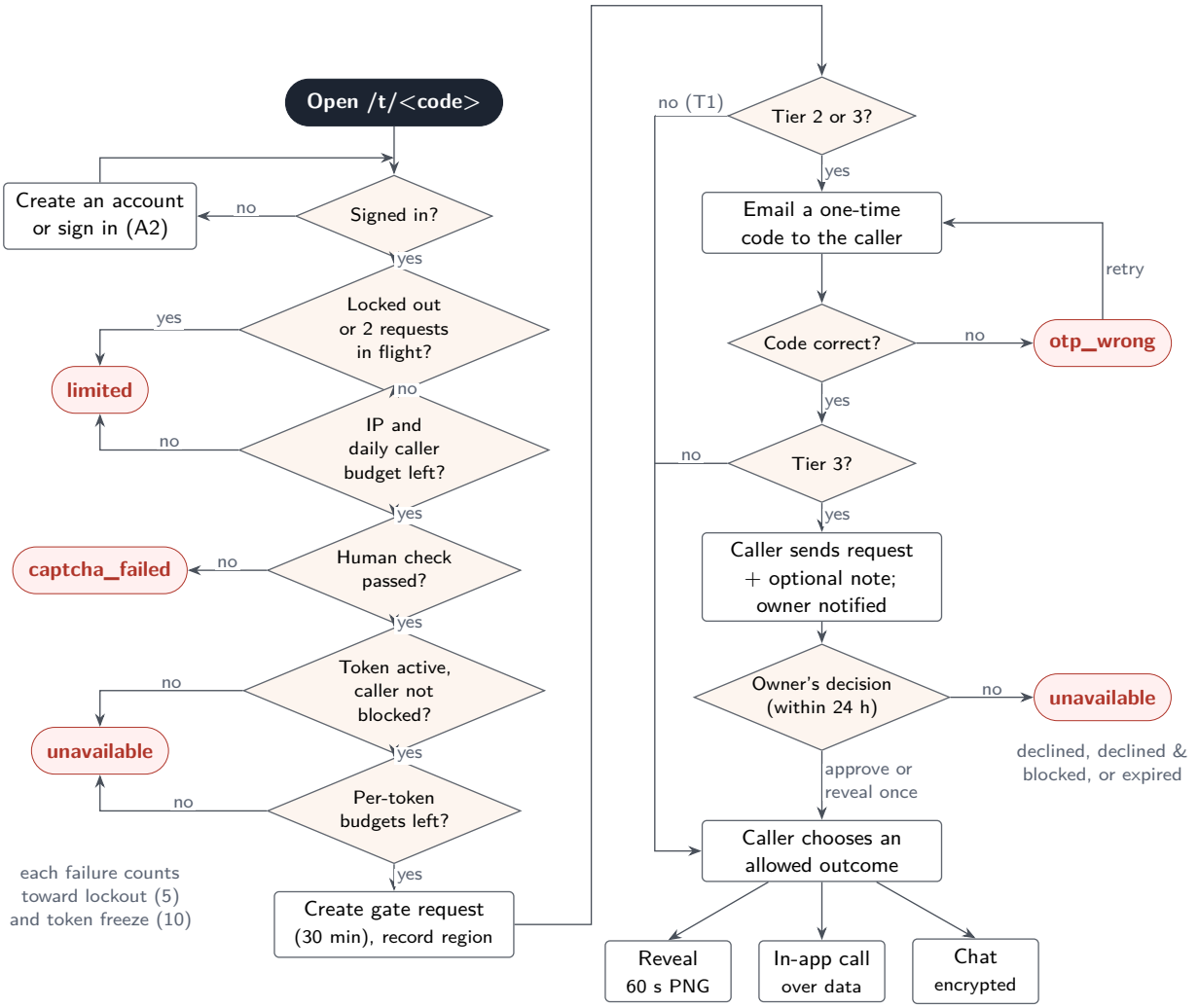



**Figure 7.1:** Decode gate decision flow

Every negative branch that involves a specific token returns the same `unavailable` body, after the same 200 ms floor. An attacker therefore can't tell an unknown code from one that is paused, revoked, frozen, used up, blocked or declined. The CAPTCHA is checked before the token is looked up, so even a failed human check reveals nothing about the code.

## 7.2 How people chat and call privately

Nobody can message or call an owner out of the blue: every contact starts from one of the owner's links or codes, and both parties' numbers stay hidden throughout.

1. **The owner shares a token.** The token's tier sets the checks; its options decide whether chat, an in-app call or a watermarked view of the number are offered. Calls are always in the app; codes without a phone number offer only chat and calls.
2. **The caller opens it and passes the gate.** They sign in (or create an account and come straight back), prove they're a person and, on Tier 2 and 3, confirm a code emailed to them.
3. **The caller chooses how to connect.** Each option is a card that explains what happens next:
   - **Chat in the app** opens an encrypted conversation. The owner gets an in-app and email notification and an unread count on **Chats**.
   - **Call in the app** rings the owner's open PryingEyes over the internet. Audio goes browser to browser through PryingEyes' own TURN relay, so no phone numbers are involved and IP addresses stay hidden. If the owner's app is closed, they get a missed call (section 7.3).
   - **Phone call through PryingEyes** (optional, when the operator enables it) rings the owner from the platform number with a whisper ("…press 1 to accept"), then rings the caller's phone and joins the two calls.
   - **Show the number** shows a watermarked image for 60 seconds.
4. **They carry on in Chats.** Replies notify the other side (at most every 15 minutes per chat, never with message text). While the caller's gate request is still valid and the token allows calls, the chat offers the caller **Call in the app**. The owner's view explains that only the person who made contact can start a call. Calls appear in the chat timeline ("In-app call · 3:12", "Missed call"). Either side can block or report at any time.

## 7.3 In-app call lifecycle

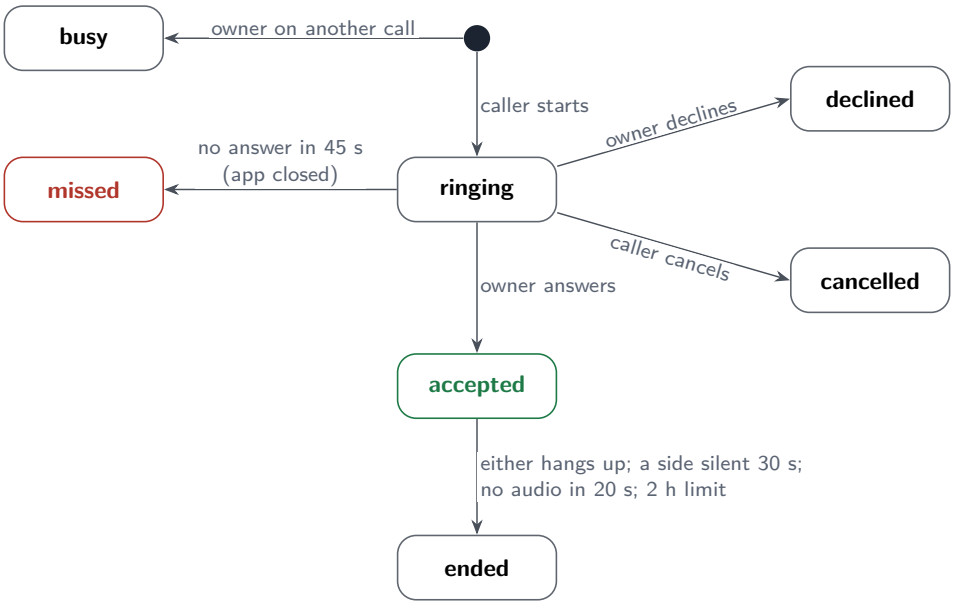



**Figure 7.2:** In-app call states. *Missed*, *busy*, and *cancelled* while the owner's app was closed all notify the owner.

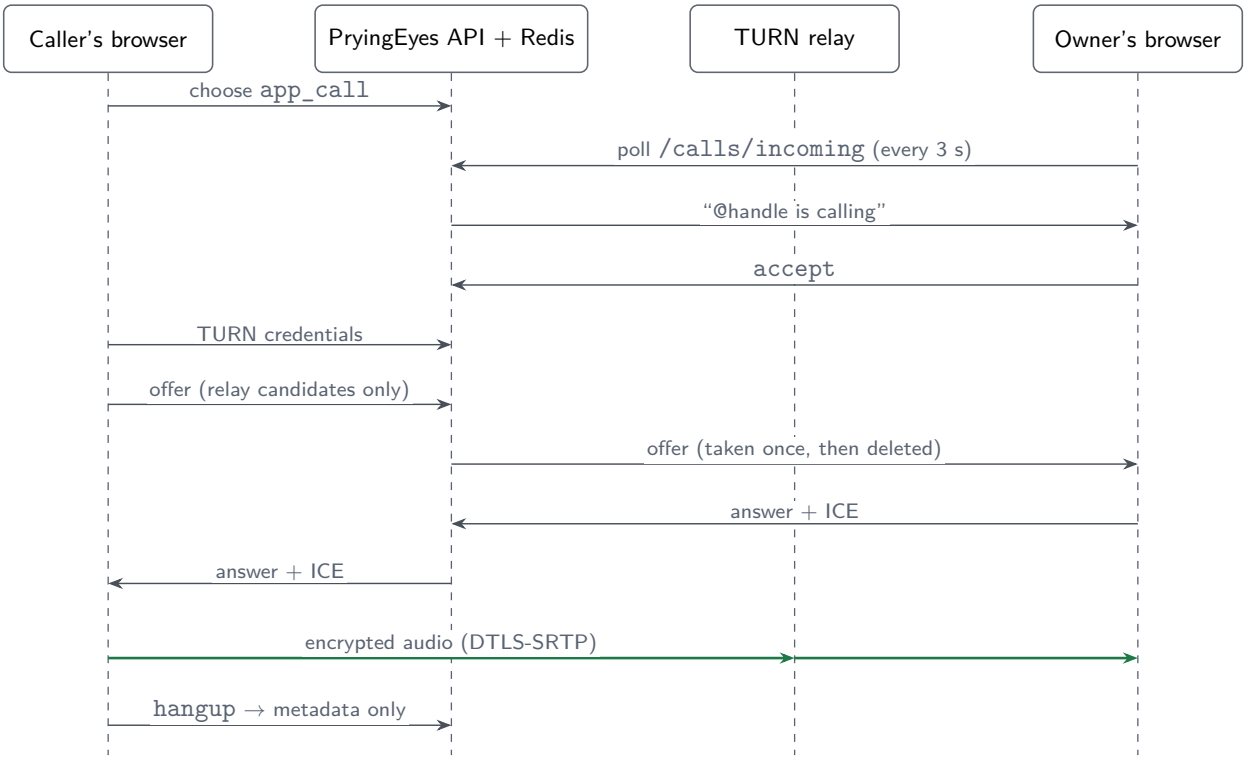



**Figure 7.3:** An answered in-app call. The API never sees audio; TURN forwards packets it cannot decrypt, and each side only sees the relay's address.

## 7.4 Native app contract

The future iOS and Android apps use the same HTTP API and state machine as the website. Only delivery of the incoming call changes:

1. **Ringing a closed app.** The server will send a VoIP push (APNs PushKit on iOS, a high-priority FCM data message on Android) from `notifyIncoming()` in `src/lib/calls.ts`. The app shows the system call screen (CallKit or ConnectionService) and, on answer, posts `accept` as the website does. If no device answers within `CALL_RING_SECONDS`, the call becomes *missed* exactly as today.
2. **Media.** Native WebRTC (libwebrtc) uses the credentials from `GET /api/calls/ice` with `iceTransportPolicy = relay`, and the same offer, answer and ICE messages through `/api/calls/[id]/signal`.
3. **Liveness.** The app keeps polling `GET /api/calls/[id]/signal` during a call, or switches to a WebSocket later. Silence for 30 seconds ends the call.
4. **Unchanged:** gate rules, who may call whom, busy and missed handling, notifications, the chat timeline, metadata-only records and audit events.

## 7.5 A complete Tier 3 contact

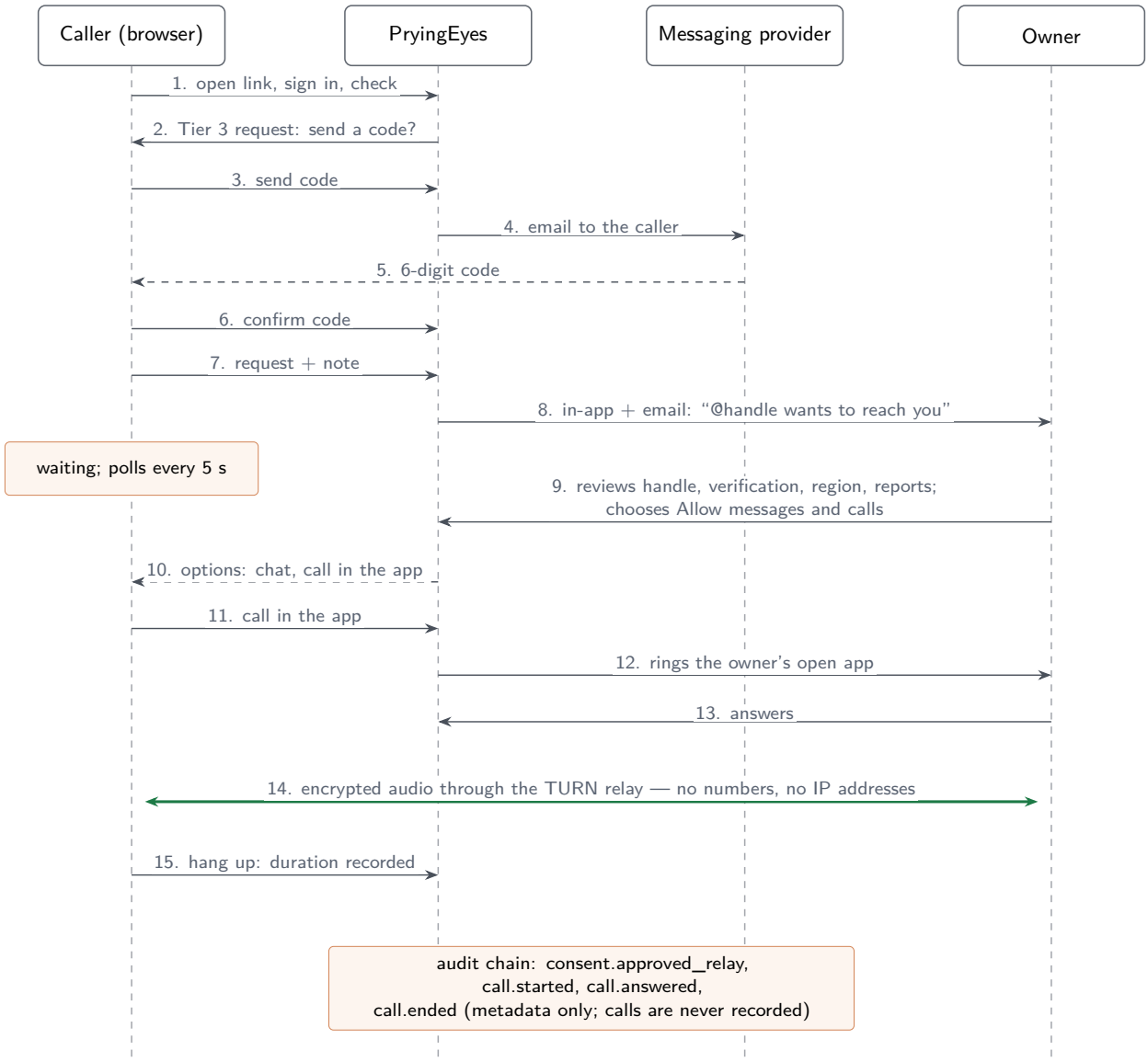



**Figure 7.4:** Sequence — Tier 3 request approved, then an in-app call

## 7.6 Token lifecycle

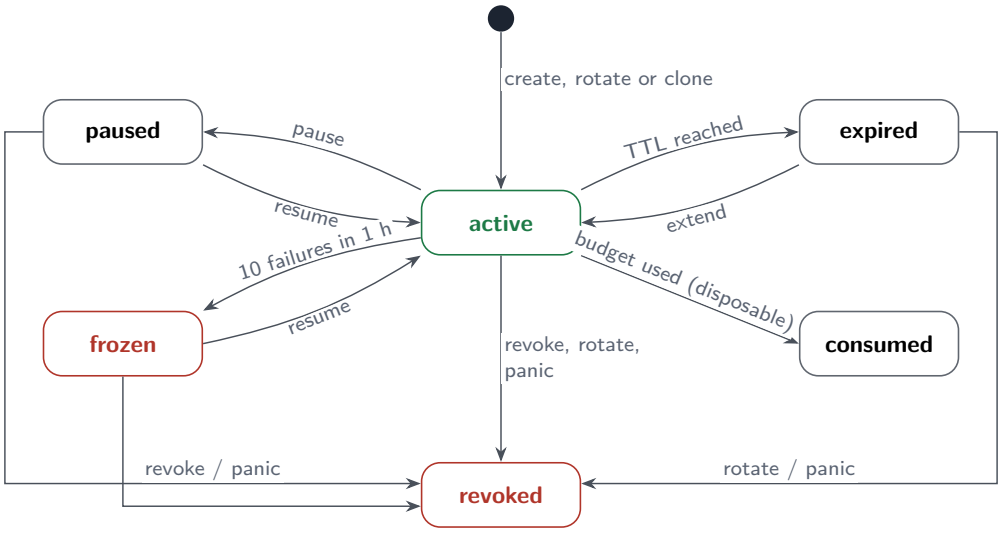



**Figure 7.5:** Token states

Only *active* tokens pass the gate. *Revoked* and *consumed* are final. A caller who already completed an outcome on a disposable token can still use the chat and call options of that same request.

## 7.7 Gate request lifecycle

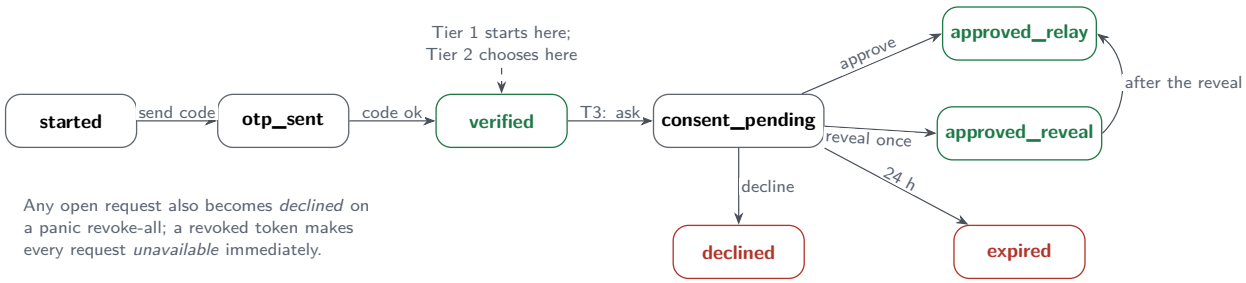



**Figure 7.6:** Gate request stages

CHAPTER 8

# Phone app

The `mobile/` folder holds the iOS and Android app, built with Expo SDK 58, Expo Router and react-native-webrtc. It uses the same server, rules and audit trail as the website. Everything a person can do on the website, they can do in the app:

- **Callers:** scan a QR code or type a code, pass the gate, then chat, call in the app or view the number. Screenshots are blocked while the number is shown.
- **Owners:** set up two-step sign-in, add numbers, create and share codes (with a QR code and the share sheet), decide requests, see notifications and activity, and manage signed-in devices.

The one difference: passkeys work on the website only; the app uses an authenticator app for two-step sign-in.

## 8.1 Running it

1. Start the server as in chapter 3: `docker compose up -d --wait`, then `npm run dev`. The compose file also runs the coturn call relay.
2. Phones and emulators can't use `localhost` to reach your computer. In the server's `.env`, set `TURN_URLS` to your computer's LAN address, for example `turn:192.168.1.20:3478?transport=udp`. Open ports 3000/tcp, 3478 and 49160–49200/udp in the firewall.
3. In `mobile/`, run `npm ci`, then pick a target:
   - **Android emulator:** `npx expo run:android`. The app reaches the server at `http://10.0.2.2:3000` automatically.
   - **iPhone simulator** (macOS): `npx expo run:ios`.
   - **Real phone:** `npx expo run:android --device` over USB, or a cloud development build with `npx eas-cli build --profile development`. On the start screen, choose **Change server address** and enter `http://<computer-LAN-address>:3000`.
   - **Expo Go:** `npm run start:go`. Everything works except calls, which need the native WebRTC module in a development build.

## 8.2 How the app authenticates

**Bearer sessions.** A sign-in request that carries `X-PE-Client: mobile` returns the session token in the response body instead of a cookie. The app stores it in the phone's keychain or keystore (expo-secure-store) and sends it as `Authorization: Bearer`. Expiry, idle timeout, remote sign-out and the device list work as on the website.

**Device binding.** The session is bound to the install's `X-PE-Device` value, the way web sessions are bound to the browser. A token copied to another device is rejected.

**Cross-site protection.** App requests carry no cookies, so there is no ambient credential for another site to abuse. The server skips the `Origin` check only for requests that carry the app header *and* no cookie. Browsers can't add that header cross-site without a CORS preflight, which production never allows.

**Client settings.** `GET /api/config` returns the human-check mode, the Turnstile site key and the development flags. It contains no secrets. In production, the app shows Turnstile in a WebView served from the site's own origin.

## 8.3 Calls in the app

The app implements the native-app contract in section 7.4:

- the same `/api/calls/*` endpoints and state machine;
- relay-only ICE settings and short-lived TURN credentials;
- the signal endpoint doubles as the heartbeat.

While the app is open, it checks for incoming calls every 3 seconds and rings on any screen with a banner and vibration. When the app is closed, the call becomes a missed call. A VoIP push (APNs PushKit, FCM) at `notifyIncoming()` will ring a closed app; nothing else changes. Calls run over data, encrypted between the two phones, at about 0.3 MB a minute.

## 8.4 Checks

| Command (in `mobile/`) | What it checks |
|---|---|
| `npm run typecheck`, `npx expo lint` | TypeScript and ESLint |
| `npm test` | Unit tests: phone-number handling, code parsing, the API client (bearer token, step-up retry, errors) |
| `npm run export:native` | Bundles the app for Android and iOS, which catches native bundling errors without a device |
| `npx playwright test -c playwright.mobile.config.ts` (repository root) | End-to-end tests on the app's web build. One test is an app caller reaching a website owner, chatting both ways, then making an in-app call through the TURN relay. The other is the full owner set-up inside the app. |

CHAPTER 9

# Safe groups and statuses

People often end up posting their phone number, name or email in a community group, or on their own status, because that's the easiest way to be reachable. Anyone in the group, and anyone they forward it to, then has it for good. PryingEyes groups and statuses remove that pressure:

- contact details can't be posted as text;
- an author who wants to be reachable attaches a **contact code**;
- tapping the code opens PryingEyes on the decode gate with the code already filled in, so nobody copies or pastes anything.

## 9.1 What people can do

| Action | Rules |
|---|---|
| Start a group | Any signed-in account. The creator is admin and gets an invite link `/g/<invite>`. Names and descriptions pass the contact-details guard. |
| Ask to join | Anyone signed in can find a *listed* group in search, or open any group's invite link, and ask to join with an optional note. Only the group's creator or an admin they appointed can let them in; admins are notified, and so is the person once approved. The invite travels in the request body, never in a query string, and nginx doesn't log `/g/` paths. |
| Post in a group | Members only. Text only (no images), up to 1,000 characters, 30 posts an hour. |
| Attach a contact code | Anyone 18+; two-step sign-in only if the code shows a phone number. The author picks the tier (default Verified) and how long it works: until switched off, 24 hours, or one person only. The same works in statuses and private chats. |
| Post a status | Visible for 24 hours to people who share at least one group with the author. Same guard, same code option. |
| Delete | Authors delete their own posts; group admins can delete any post in their group. |
| Report | Goes to the moderation queue with the post text attached, and notifies the group admins. Optionally blocks the author, which hides their posts both ways. Moderators can **Remove post**. |
| Manage members | Admins approve or decline join requests, choose whether the group shows in search, reset the invite link (the old one stops working), remove members and make admins. The last admin to leave hands over to the longest-standing member. |

## 9.2 The contact-details guard

`src/lib/guard.ts` runs on every group name, description, post and status, on the server. It refuses:

- **phone numbers**, found with libphonenumber for the default country, plus any run of seven or more digits separated only by spaces, dots, dashes or brackets;
- **email addresses**, including *name at gmail dot com* and *[at]/[dot]* forms;
- **messaging links**: wa.me, t.me, signal.me, m.me, `tel:`, `sms:`, `mailto:`.

Before checking, it rewrites disguises: digit words (*seven six one two*, *double seven*), full-width digits, and O or l used as 0 or 1 inside numbers. Dates, times, prices and PryingEyes codes are allowed. A refusal returns `contact\_info\_detected` with what was found and a pointer to contact codes; the apps then switch on **Attach my contact code**. Refusals are audited as `post.rejected\_contact`, without the text.

The guard is strict on purpose. A false positive costs a rewording; a false negative publishes someone's number.

## 9.3 Contact codes on posts

- Attaching a code mints a token from the author's protected number, labelled *Group: <name>* or *Status updates*. Later posts in the same place reuse it while it's active.
- Each group therefore has its own code. The owner sees where each request came from, and revoking one group's code on the dashboard switches it off in that group only. Posts then show *This contact code has been switched off.*
- Post text and the attached code are encrypted under the author's data key, so erasing the account erases them.
- Members see the short code and a link `https://<host>/t/<code>`. The decode gate applies exactly as for any other code: human check, the tier's checks, budgets, audit.

## 9.4 Links that open the app

**Universal and App Links.** Builds made with `EXPO\_PUBLIC\_API\_URL=https://<host>` declare the host for `/t/*` and `/g/*`: an Android intent filter with `autoVerify`, and the iOS associated domain. The server publishes `/.well-known/assetlinks.json` (from `ANDROID\_CERT\_SHA256`) and `/.well-known/apple-app-site-association` (from `IOS\_TEAM\_ID`). The phone then opens the app straight on the gate or the join screen.

**Custom scheme.** `pryingeyes://t/<code>` and `pryingeyes://g/<invite>` work in every build. The website's **Open in the app** banner on phones uses them.

**Sign-in in between.** If the person isn't signed in, the code or invite is kept through sign-up and sign-in, and they land back on it.

**Paste.** For codes copied from elsewhere, the app's **Paste** button reads the clipboard and goes straight to the gate.

## 9.5 API

| Endpoint | Purpose |
|---|---|
| `GET\|POST /api/groups` | List my groups; create one |
| `POST /api/groups/join` | `\{invite, preview?, note?\}`: preview, or ask to join by invite |
| `GET /api/groups/discover?q=` | Search listed groups |
| `POST /api/groups/:id/join` | `\{note?, cancel?\}`: ask to join a listed group, or withdraw |
| `GET\|POST /api/groups/:id/requests` | Admins: list requests; `\{userId, decision: approve\|decline\}` |
| `GET\|PATCH /api/groups/:id` | Details, members, invite (admins); rename |
| `POST /api/groups/:id/invite` | New invite link |
| `POST /api/groups/:id/leave` | Leave |
| `POST /api/groups/:id/members` | `\{userId, action: remove\|make\_admin\}` |
| `GET\|POST /api/groups/:id/posts` | Feed; post `\{text, attach?: \{numberId?, tier, kind?\}\}` (`kind`: standard, day, one) |
| `POST /api/chats/:id/messages` | `\{text, attach?\}`: a message, optionally carrying a fresh contact code |
| `GET\|POST /api/status` | Status feed grouped by person; post a status |
| `DELETE /api/posts/:id` | Delete (author or group admin) |
| `POST /api/posts/:id/report` | `\{reason, details?, block?\}` |

The tables `groups`, `group\_members` and `posts`, and the column `reports.post\_id`, come from migration `drizzle/0003_social.sql`. Expired statuses are deleted by the retention job.

## 9.6 Codes without a phone number

Since version 0.6.0 a code doesn't need a phone number behind it, and calls never use the phone network.

- **App-only codes** (the default) have no number. Callers who pass the gate can **Send a message** or **Call in the app** over data, through the TURN relay. Nobody's number or IP address is shown.
- **Anyone can share.** An account that is verified and 18+ can create app-only codes, attach them in groups, take Tier 3 requests and see activity straight away. Two-step sign-in isn't required, because no number is stored.
- **Showing a number is optional.** An owner can add a number (**Settings** → **Show my number**) after setting up two-step sign-in, then pick it when creating a code. Callers on that code may then see it once, as a 60-second watermarked image. Creating, renewing or replacing such a code, and approving **Show my number once**, need two-step sign-in. Pausing or revoking never does.
- **Email only (0.7.0).** Accounts sign up with email and password; no text messages are sent. Tier 2 and 3 codes are emailed. People see the handle, never the email.
- Gate options are therefore `chat` and `app_call`, plus `reveal` only on a code with a number. Migration `0004_app_only_codes` makes `tokens.number\_id` optional; `0005_drop_phone_relay` removes the old phone-call table.

## 9.7 Codes shared anywhere

Contact codes are meant to be passed around. A code can be attached to a group post, a status or a private chat message, and any code or code link typed into a post or a chat becomes tappable. Tapping one opens the owner's decode gate with the code already filled in; from there the caller messages or calls the owner in the app.

| How long | Behaviour |
|---|---|
| Until switched off | Reused for later posts in the same group (or the status feed); the owner revokes it on **My codes**. |
| 24 hours | A new code each time; it expires a day later, like a status. |
| One person | A new code each time; it stops after the first person gets through the gate. The card then says it has been used. |

A code shared in a chat is labelled *Shared with @handle*, so the owner can see on **My codes** and in Activity where a contact came from.

CHAPTER 10

# The system's screens

These are real screenshots of PryingEyes, taken by the script `tests/screens/capture.spec.ts`. It walks through the system as real people would: an owner creates codes and a group, a caller signs up from a code, they chat, a code is shared in the chat, a call rings, someone asks to join a group and an admin lets them in. Re-take them at any time with:

```
cd mobile && npx expo export --platform web --output-dir dist-web && cd ..
npx playwright test -c playwright.screens.config.ts
```

Website screens are $1280 \times 800$ (desktop). Phone app screens are $412 \times 915$, a typical Android phone; they come from the app's web build, which renders the same React Native screens as the installed app. Handles such as `@quiet\_heron\_45` are generated, as they would be for real people.

## 10.1 Look and feel

Since version 0.9 the website and the phone app share one visual style, taken from the *Talk Sense* social app concept on Dribbble. The same values are defined twice, in `src/app/globals.css` for the website and `mobile/src/ui/theme.ts` for the app, so the two stay in step.

| Element | Treatment |
|---|---|
| Background | A soft blush-to-lilac "sky" gradient (`\#F6E8EF` to `\#E8D8EC`); a deep plum version in dark mode. |
| Cards | Frosted, translucent white with a 22-pixel radius and a faint pink shadow. |
| Type | Merriweather (serif) for page titles, names, codes and the call timer; Inter (sans-serif) for everything you read and type. Both are bundled with the app and the site, so no font is fetched from another server. |
| Buttons | Ink-black pills (`\#1C1A1F`) with white text for the main action; white pills with a thin border for the others. |
| Accent | Magenta (`\#B5166A` for text, `\#E0218A` for badges and highlights), chosen to pass WCAG AA contrast on white. |
| Chips and choices | White pills; the selected one turns ink-black. |
| Chat | Rounded bubbles: yours ink-black, theirs white, with the time tucked inside. The composer is a round code button and a pill-shaped "Type a message here…" field. |
| Status avatars | Rounded squares rather than circles, ringed in magenta when there is something new. |
| Calls | A plum-to-ink panel with a pale pink serif timer and round controls. |
| Icons | One set of line icons (24-pixel grid, 1.75 stroke) on both the website and the app. *Send* and *Post* are a see-through magenta paper plane; *report* is a flag, *block* a struck circle, *delete* a bin, *sign out* an exit arrow; calls use round green and red buttons. Icon-only buttons keep a spoken name (for example "Send"), so screen readers and the tests still find them; actions whose meaning needs words, such as *Let in* or *Leave group*, keep their label beside the icon. |

## 10.2 Website

## 10.3 Phone app

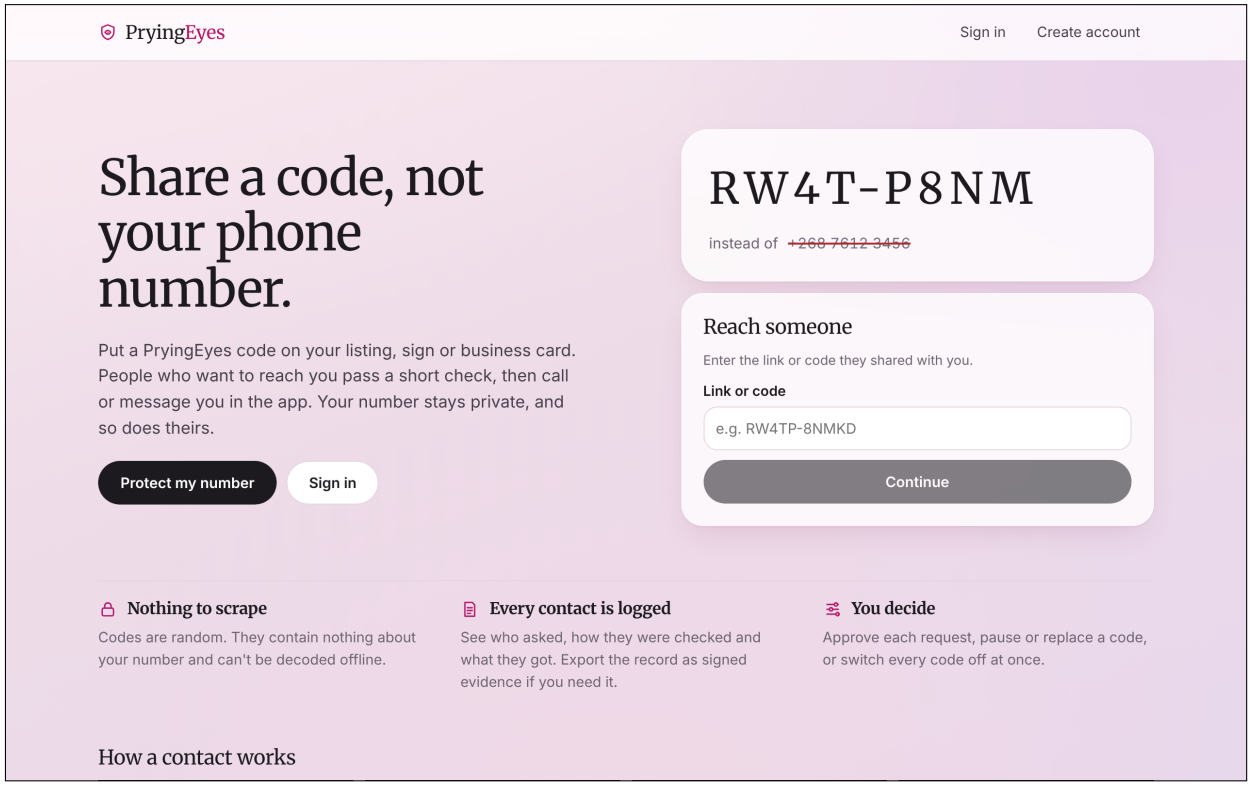



**Figure 10.1:** Home page: share a code, not your phone number

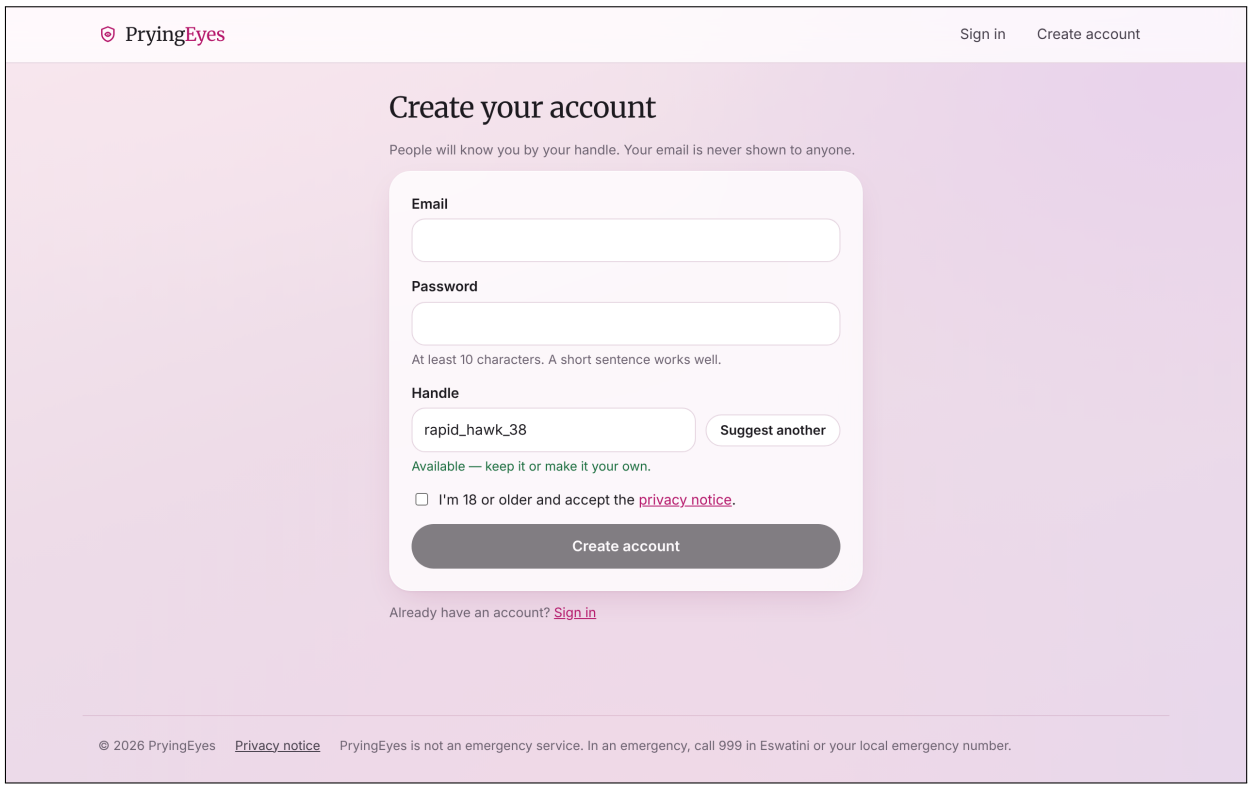



**Figure 10.2:** Sign-up: email and password, with a generated handle (nobody sees the email)

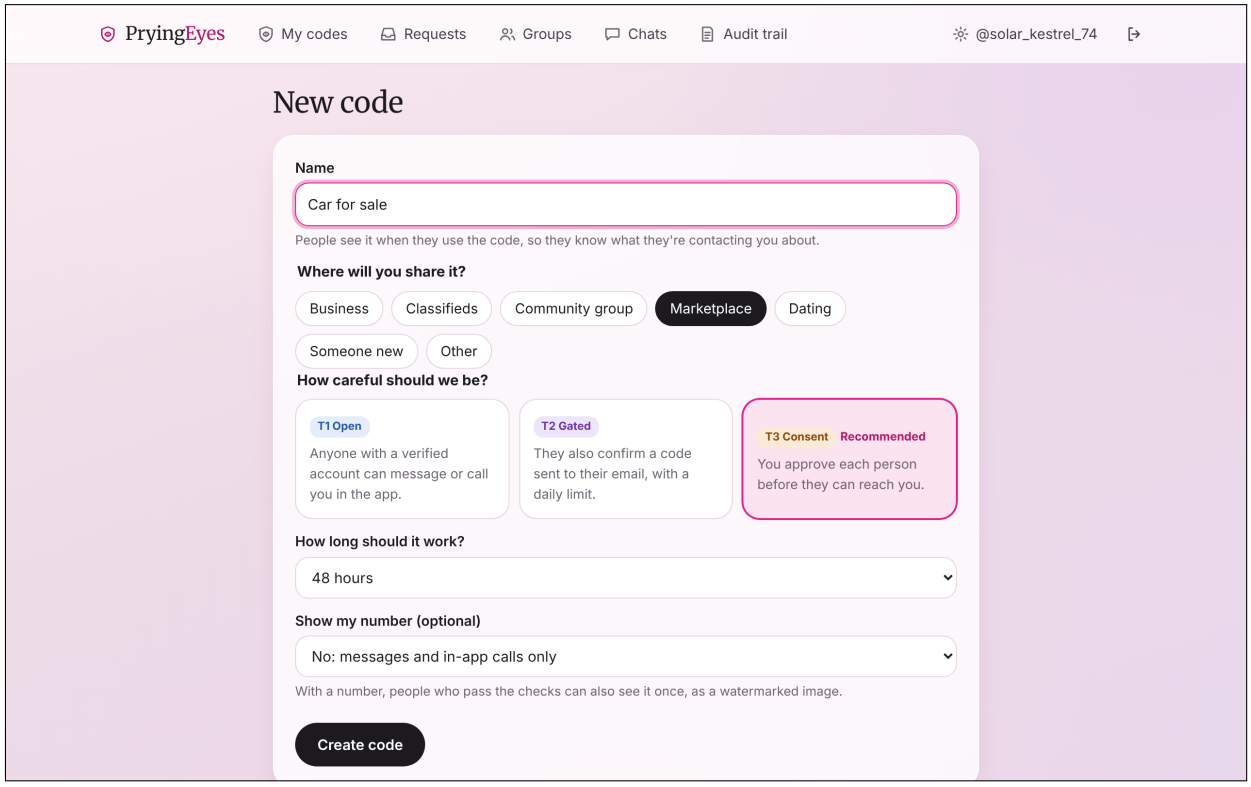



**Figure 10.3:** New code: a name, where it will be shared, how careful to be, and how long it works

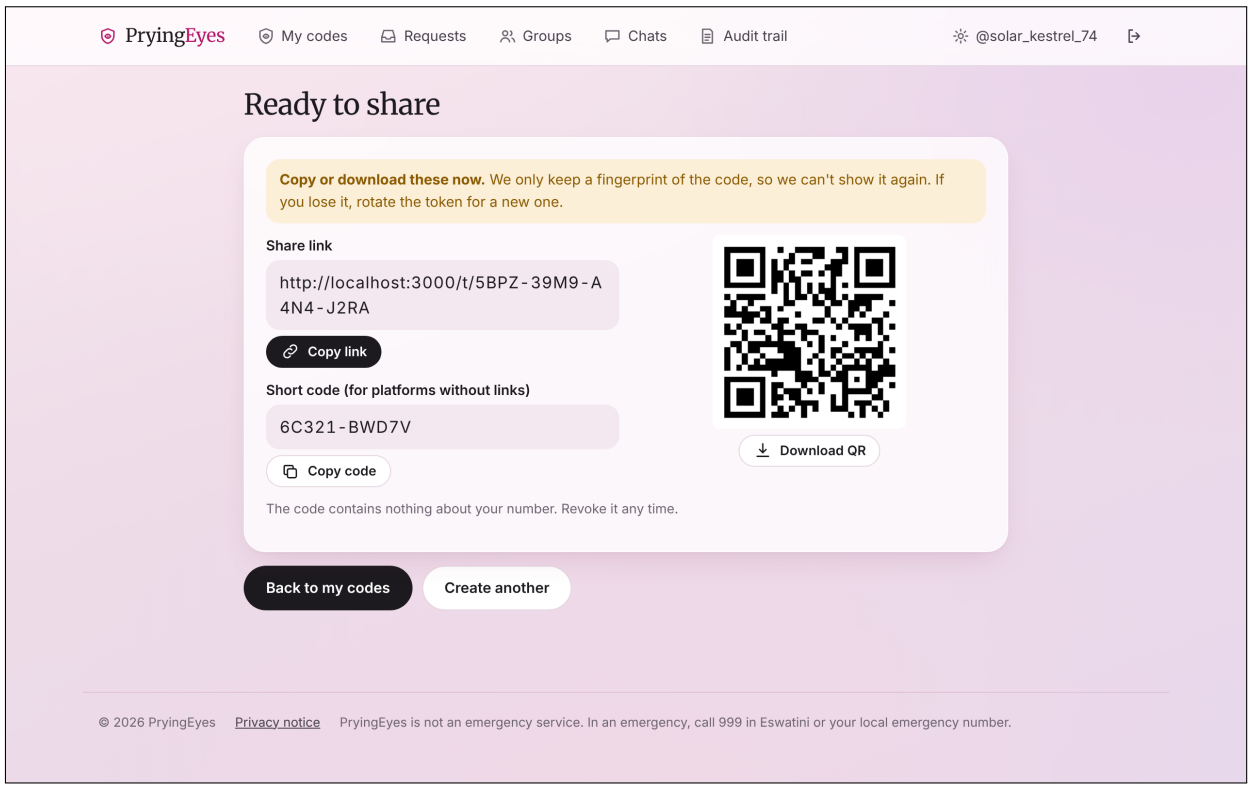



**Figure 10.4:** A new code, ready to share as a link, a short code or a QR code

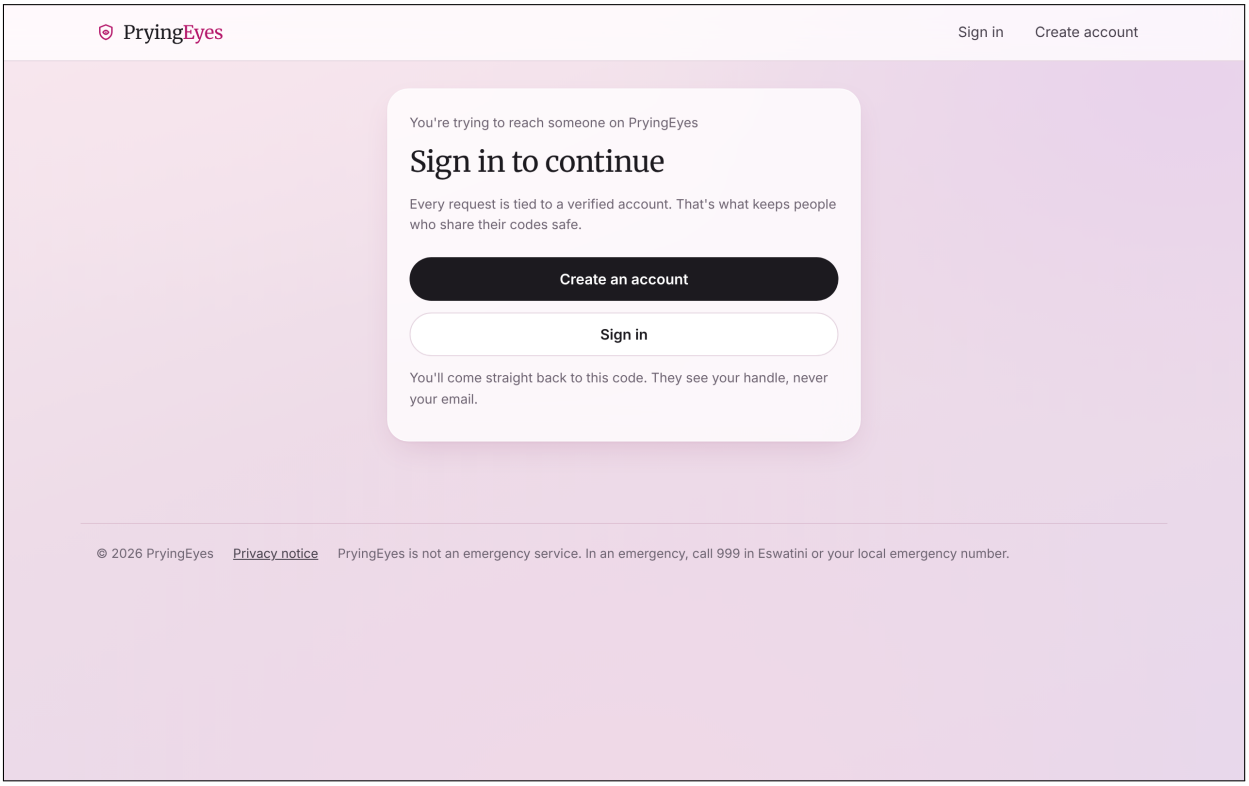



**Figure 10.5:** A caller opens a code while signed out: create an account or sign in, then come straight back to it

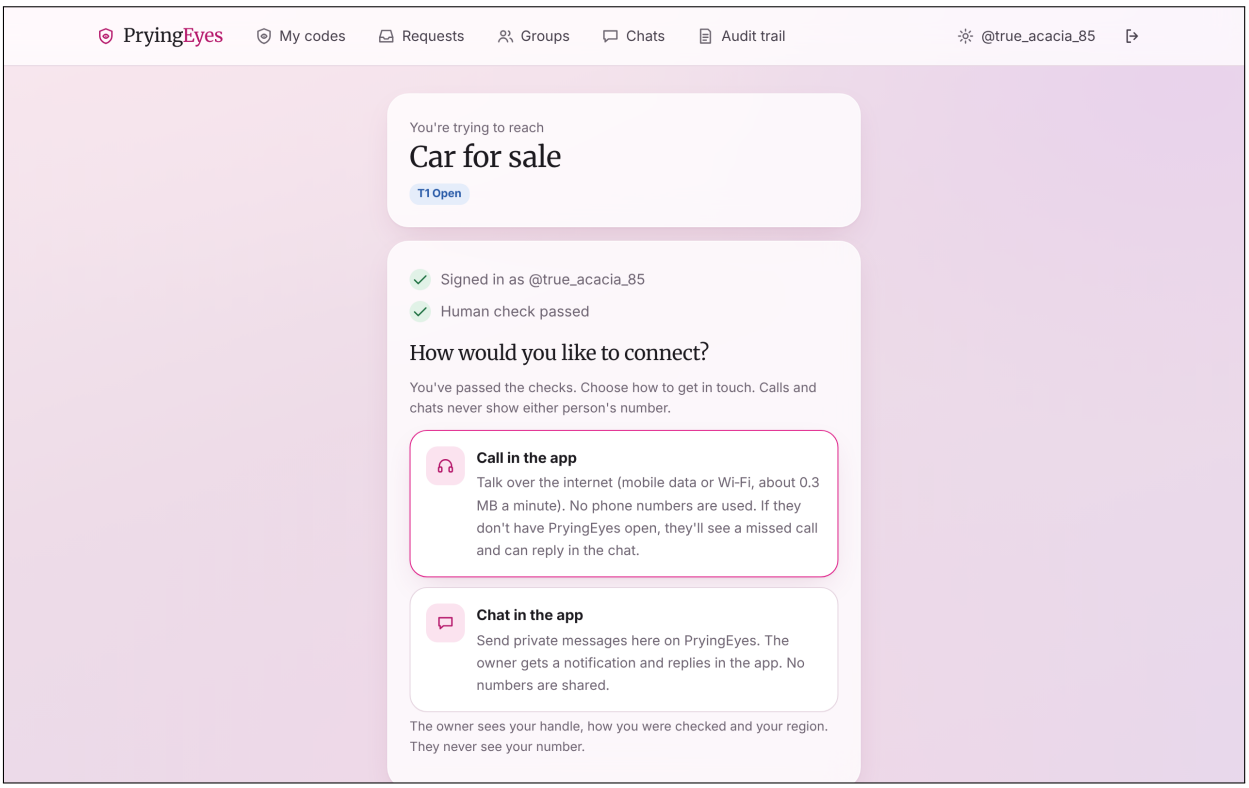



**Figure 10.6:** The gate after the human check: message or call the owner in the app

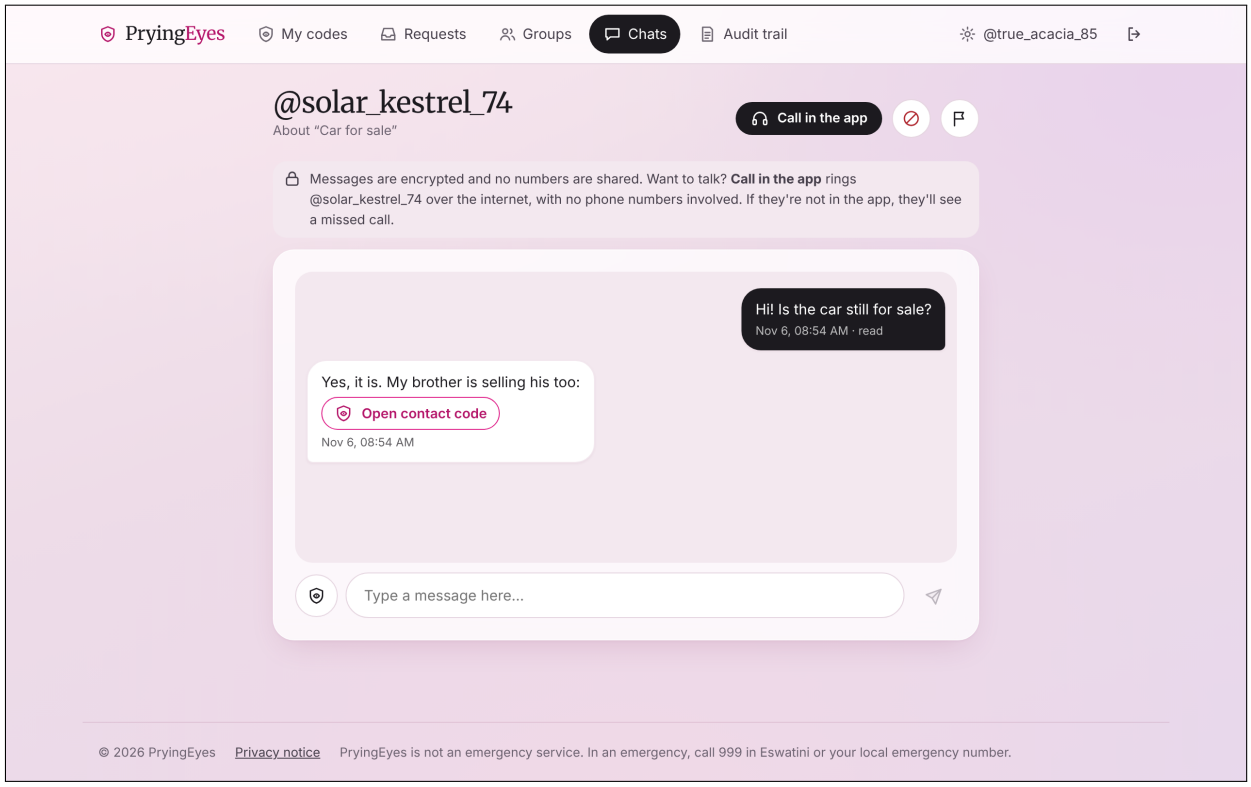



**Figure 10.7:** Private chat. The owner has shared a one-person code, shown as **Open contact code**; tapping it opens that code's gate

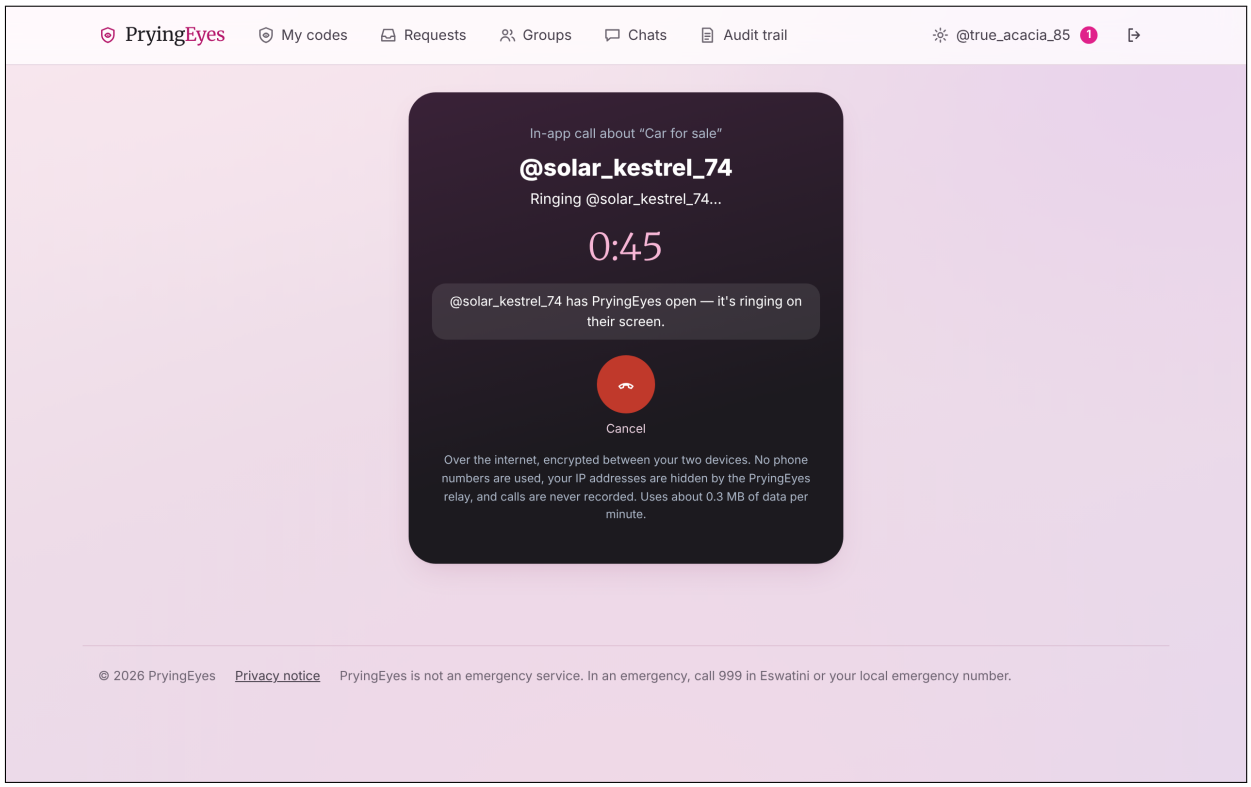



**Figure 10.8:** An in-app call ringing: over the internet, through the TURN relay, with no phone numbers

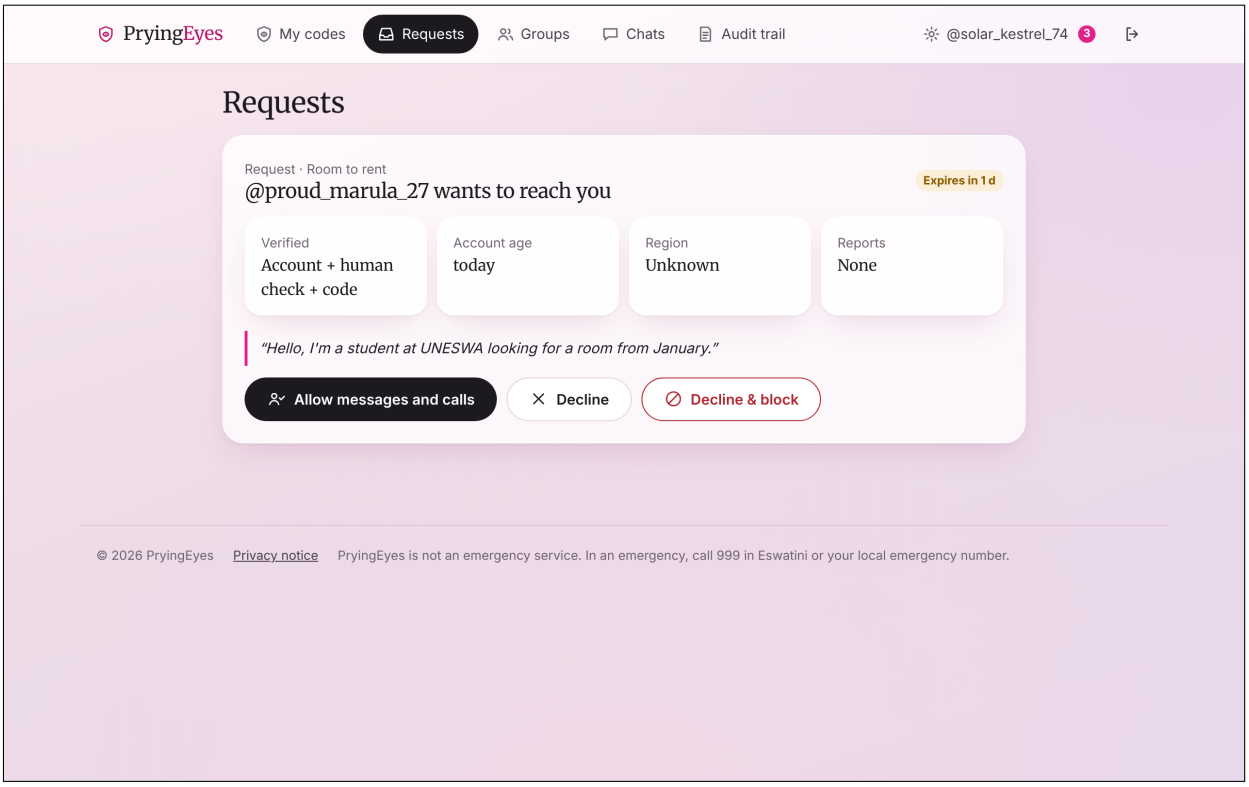



**Figure 10.9:** Tier 3: the owner sees who is asking, how they were checked, and their note, then decides

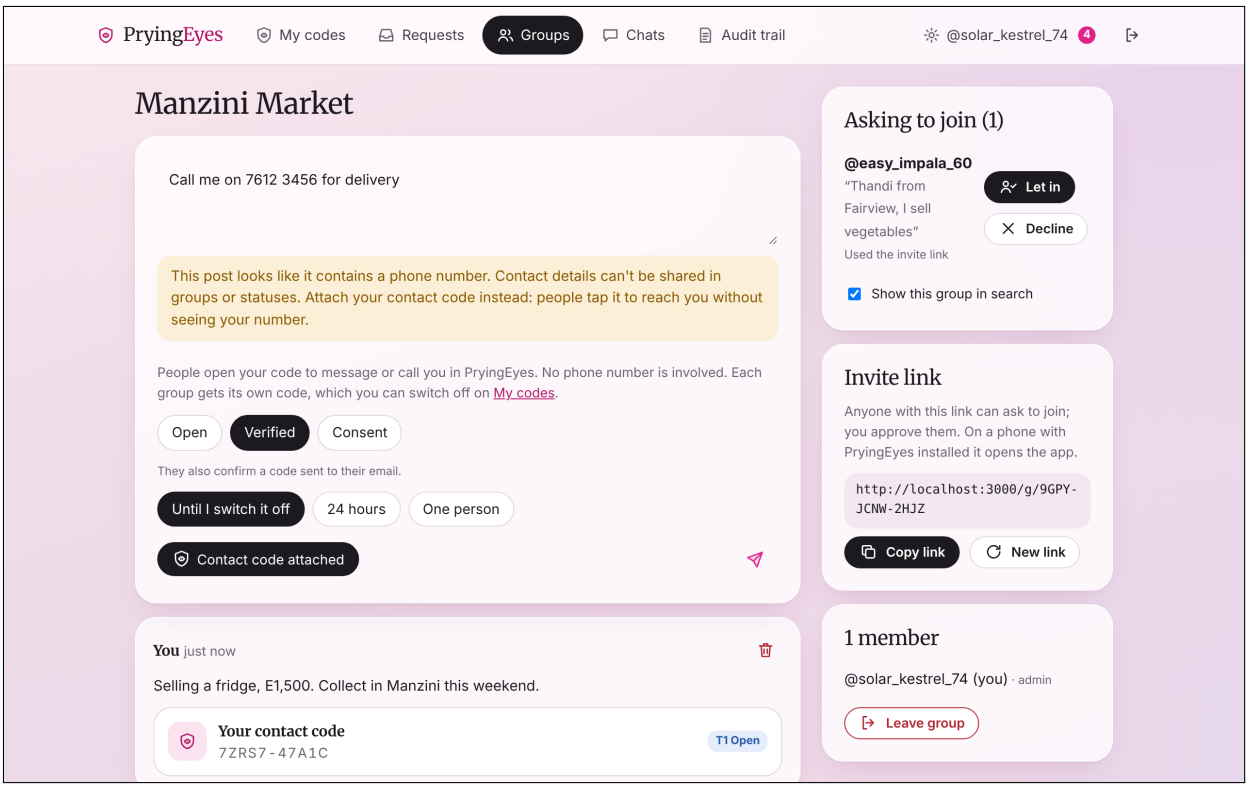



**Figure 10.10:** A group as its admin sees it: a post with a contact code, a phone number refused by the guard, and someone asking to join

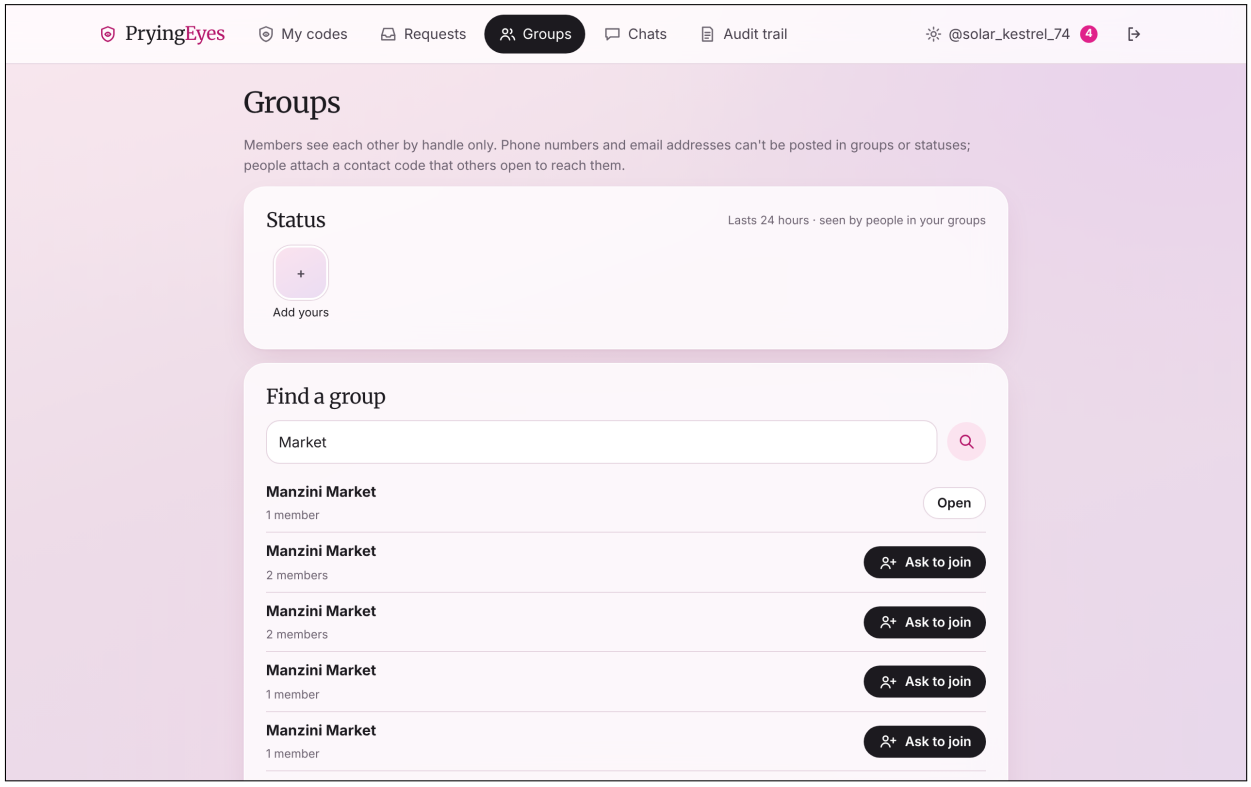



**Figure 10.11:** Groups: 24-hour statuses, finding a group to ask to join, and your groups

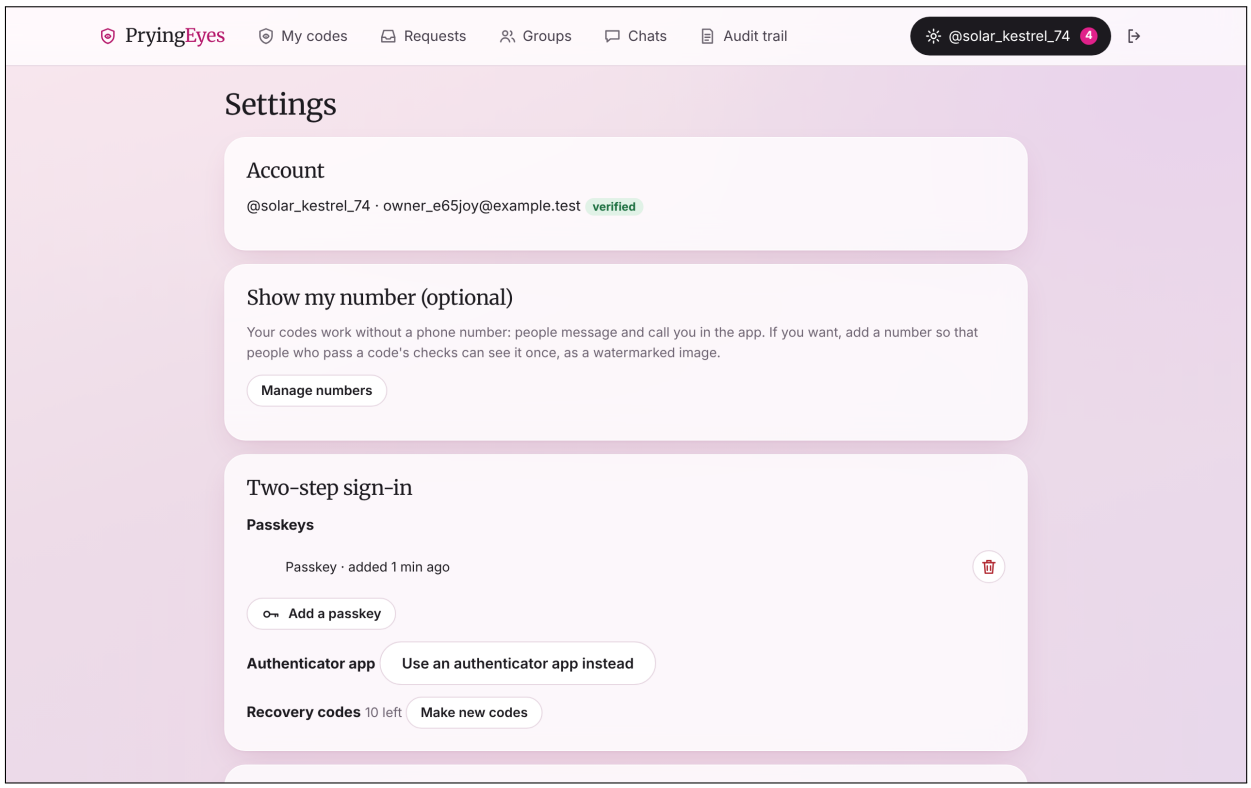
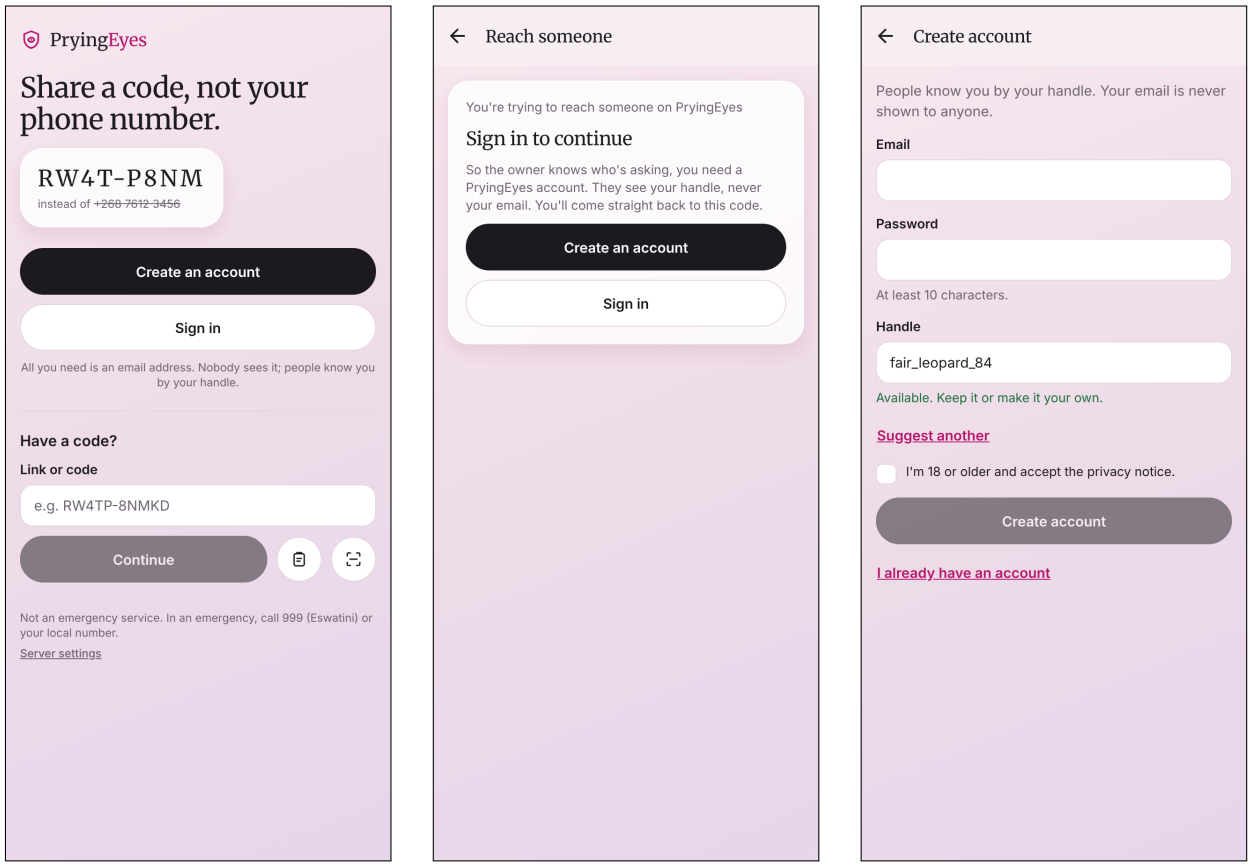



**Figure 10.12:** Settings: account, the optional **Show my number**, two-step sign-in and signed-in devices

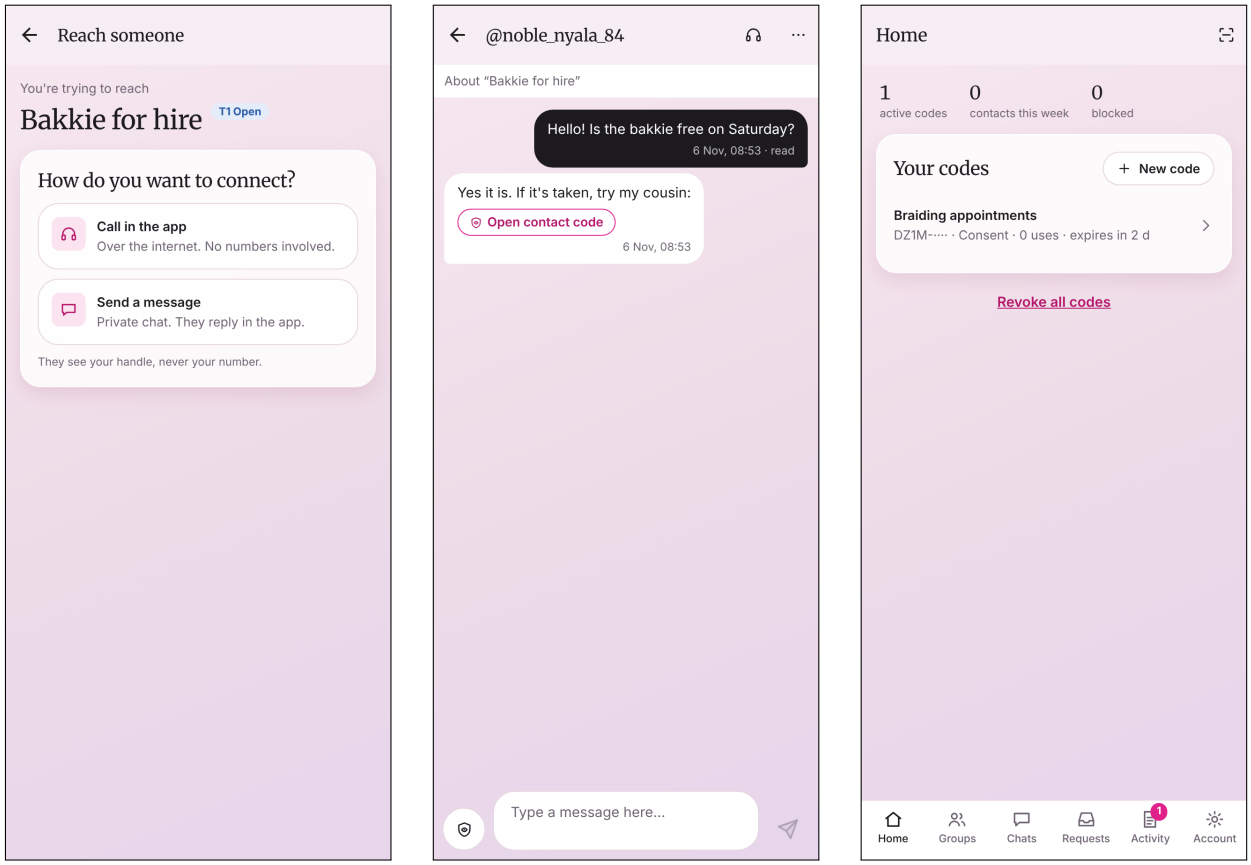



Start: create an account, sign in, or enter a code

A code opened while signed out

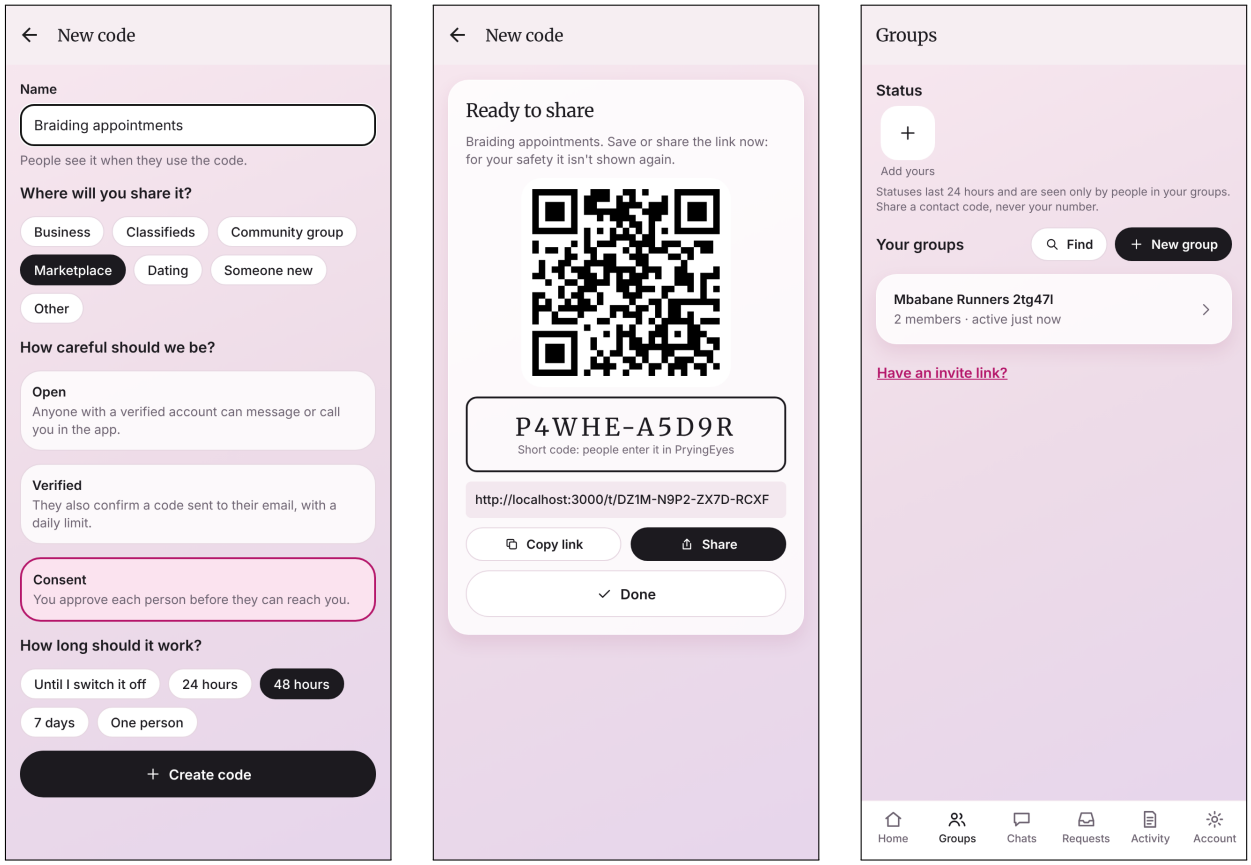
Email sign-up, carrying the code along

**Figure 10.13:** Phone app: getting in



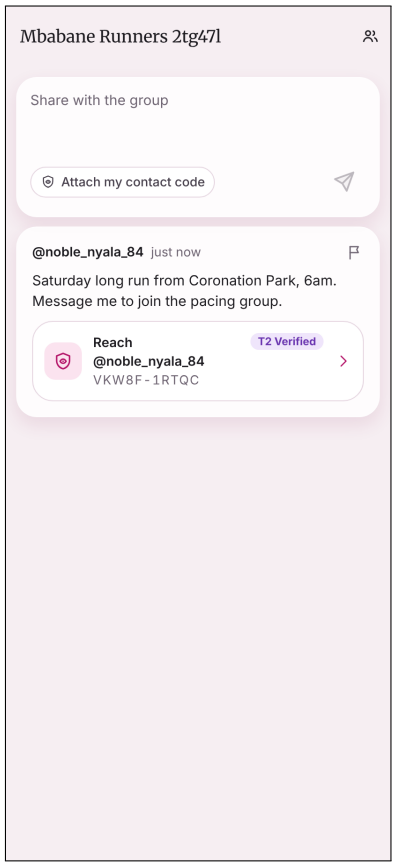

Back on the code: message or call | Chat with a shared code | Home: your codes

**Figure 10.14:** Phone app: reaching someone, and your own codes

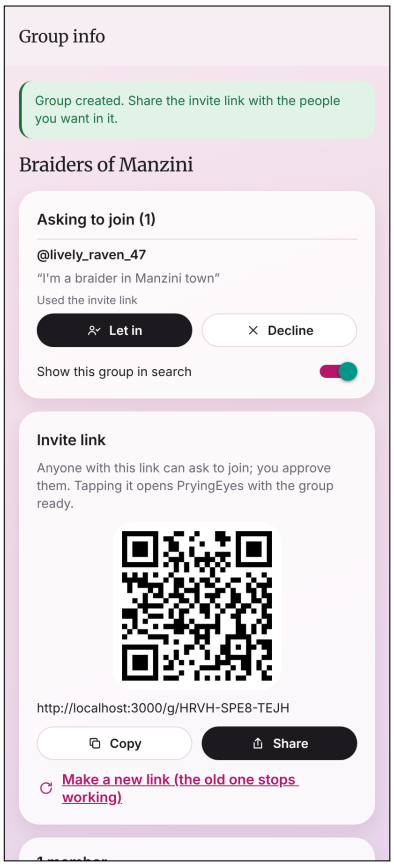



New code Ready to share Groups and statuses

**Figure 10.15:** Phone app: making and sharing a code




A group feed with a contact code




Group info: someone asking to join

**Figure 10.16:** Phone app: groups

CHAPTER 11

# Operations, testing and troubleshooting

## 11.1 Automated tests and acceptance checks

**Table 11.1:** Test suites and acceptance checks

| Check | Command | Pass condition and last result |
|---|---|---|
| Unit + integration | `npm test` | All green. **106/106** (23 unit; 83 against real PostgreSQL and Redis). |
| Browser journeys | `npm run test:e2e` | **8/8**: passkey onboarding; handle box; Tier 1 reveal; Tier 3 consent, chat and panic; chat; passkey sign-in; accessibility; mobile layout. |
| Enumeration campaign | `npm run attack:enumerate` | Over 99.9% of 10,000 random guesses blocked and none succeed. **99.95%, 0 found.** |
| Plaintext numbers | `npm run scan:plaintext` | No stored number, in any format, in any table, log or extra file. **0 findings.** |
| Audit chain | `npm run audit:verify` | Every link and anchor verifies. **Pass.** |
| Load | `npm run load:test` | Gate p95 under 300 ms at 50 requests per second. **p95 222 ms.** |
| Dependencies | `npm audit -omit=dev` | No known vulnerabilities in production dependencies. **0.** |
| Everything above | `npm run verify:all` | Prints a pass/fail summary (about 5 minutes). |

### How to run them

```
npm test                          # uses the separate pryingeyes_test database
npx playwright install chromium   # once
npm run build && npm start        # terminal 1
npm run test:e2e                  # terminal 2
npm run verify:all                # or everything, in order
```

The test database `pryingeyes_test` is created by the Docker setup and wiped at the start of every test run. Never point `TEST_DATABASE_URL` at real data.

The load test needs the gate budgets raised, or the budgets themselves would throttle it. Start the server like this first:

```
RL_IP_PER_HOUR=10000000 RL_CALLER_PER_DAY=10000000 \
RL_CALLER_TOKEN_PER_DAY=10000000 RL_TOKEN_PER_DAY=10000000 npm start
```

## 11.2 Running in production

`docs/DEPLOY.md` is the full guide. In summary:

1. Set `APP_ENV=production` and an HTTPS `APP_URL`, configure real providers (Turnstile and Resend), and use managed PostgreSQL 16 and Redis 7.
2. Terminate TLS 1.3 at nginx. Set `TRUST_PROXY_HOPS` to match the proxy chain. Keep `/t/...` and `/g/...` out of access logs; an nginx map is provided.
3. Run two processes: `npm start` and `npm run jobs`. Alternatively use `docker compose -profile app`.
4. Put `AUDIT_ANCHOR_DIR` on write-once storage (S3 Object Lock or `chattr +a`) and run `npm run audit:verify` daily.
5. Replace the local key provider with a cloud KMS before launch.
6. Complete `docs/LAUNCH_CHECKLIST.md`. The blockers are legal review (including registration with ESCCOM under the Eswatini Data Protection Act 2022) and an independent OWASP ASVS Level 2 penetration test.

## 11.3 Email with Resend

1. In Resend, add and verify your sending domain. Add the SPF and DKIM records it shows to your DNS, plus a DMARC record.
2. Create an API key with *sending access* only, restricted to that domain if possible.
3. Set `EMAIL_PROVIDER=resend` and `RESEND_API_KEY`, and set `EMAIL_FROM` to an address on your domain, for example `PryingEyes <no-reply@yourdomain.com>`. `EMAIL_REPLY_TO` is optional. In staging and production the app refuses to start with the `@resend.dev` test sender.
4. Run `npm run email:test -- --to=you@yourdomain.com` and check the Resend dashboard. The test inboxes `delivered@resend.dev` and `bounced@resend.dev` simulate outcomes.

How the app uses Resend:

- Each email is sent once, with a fresh idempotency key that is reused if it has to retry, so a retry can't duplicate it.
- Only 429 and 5xx responses are retried, up to 3 attempts with back-off (`Retry-After` is honoured). Errors such as an invalid key or unverified sender fail at once.
- Every email has a plain-text body and a matching minimal HTML copy, and is tagged by category (`verification`, `one_time_code`, `consent_request`, `token_frozen`, `security`, ...) for filtering in Resend.
- Logs record the Resend email id, category and any error name. Addresses and content are never logged.
- If the confirmation email fails, sign-up answers `email_unavailable`; signing up again replaces the unconfirmed attempt and sends a fresh link. A failed one-time code answers `delivery_failed`.

## 11.4 Data retention schedule

**Table 11.2:** Retention, applied by the housekeeping worker

| Data | Kept for |
|---|---|
| Audit event payloads | Owner's choice: 1–24 months, default 12. Events with no owner: 12 months. Hashes are kept forever so the chain still verifies. |
| Tier 3 chat messages | 24 hours |
| Other chat messages | Until either party erases their account |
| One-time codes, human-check results | 90 days |
| Email verification links | 90 days after expiry |
| Reveal grants | 1 day after expiry (each is usable for 60 seconds) |
| Sessions | 1 day after expiry; 30 days after sign-out |
| Tier 3 intro notes | 30 days after the request closes |
| Development outbox | 7 days |
| Pending admin requests | Expire after 24 hours |
| Erased accounts | Immediately from live data; from backups within 35 days, and unreadable at once because the data key is deleted |

## 11.5 Backups and recovery

- Back up PostgreSQL nightly, encrypted, keeping copies for no more than 35 days so erasure reaches backups.
- WAL archiving gives a 5-minute recovery point; the target recovery time is 4 hours. Test a restore every quarter.
- Back up `.env`, or your secrets manager, separately and securely. Without the same keys, a restored database can't decrypt numbers.
- Redis holds only counters and short-lived challenges and needs no backup.

## 11.6 Monitoring

Alert on:

- 5xx error rate;
- p95 latency of `/api/gate/start` above 300 ms;
- any failure of `npm run audit:verify`;
- failures of the housekeeping worker;
- Resend bounces and complaints;
- moderation reports older than 60 minutes without a response.

Logs are JSON lines (pino) with ISO timestamps. Phone numbers, token paths and secrets are redacted before writing.

## 11.7 Troubleshooting

**Table 11.3:** Common problems

| Symptom | Cause | Fix |
|---|---|---|
| `db:migrate` fails with "Connection terminated unexpectedly" | PostgreSQL was still initialising on its first start. | Run it again; the current script retries for 30 seconds. Use `docker compose up -d -wait`. |
| `docker compose up` fails: port 5433 or 6380 in use | Another service is on that port. | Change the left-hand port in `docker-compose.yml` and the matching URL in `.env`. |
| `EADDRINUSE :3000` | Another dev server is running. | Stop it, or run `npx next dev -p 3001` and set `APP_URL` to match. |
| "Too many attempts" while testing | Rate limits and lockouts work as designed. | Clear the counters (below), or wait. |
| Passkey button does nothing, or "couldn't verify that passkey" | WebAuthn needs `localhost` or HTTPS, and `RP_ID` must match the host. | Use `http://localhost:3000`, not an IP address. On a LAN, use an authenticator app instead. |
| No email arrives | The development provider captures emails. | Open `/dev/outbox`. For real emails, see §11.3. |
| With Resend, emails reach only you | The test sender `onboarding@resend.dev` only delivers to the Resend account owner. | Verify your domain in Resend and set `EMAIL_FROM` to an address on it. |
| Sign-up says "We couldn't send the confirmation email" | Resend refused or couldn't be reached (bad key, unverified sender, outage, or a rate limit). | Run `npm run email:test`. The log shows Resend's error name, never the address. Signing up again sends a fresh link. |
| "Invalid configuration: RESEND_API_KEY" | `EMAIL_PROVIDER=resend` without a key. | Add the key, or use `EMAIL_PROVIDER=outbox` locally. |
| Lost a token's link | Only a fingerprint is stored. | Use **Rotate** to get a new link; the old one stops working. |
| App refuses to start: "Unsafe production configuration" | `APP_ENV` is staging or production while a development stand-in or http URL is configured. | Configure real providers and HTTPS, or set `APP_ENV=development` for local use. |
| "Invalid configuration: MASTER_KEK ..." | Keys missing or malformed. | Run `npm run keys:generate`. |
| Stored numbers can't be decrypted after an update | `.env` keys changed. | Restore the original `.env`. Keys must never change on a system with data. |

| Symptom | Cause | Fix |
|---|---|---|
| Sign-up says "Handle: use 3–24 letters…" | The handle contains something not allowed, e.g. an email the browser autofilled, a hyphen, or more than 24 characters. | Tap **Suggest another**, or use the corrected handle the form offers. |
| `npm ci`: "18 moderate vulnerabilities" | Advisories in development-only tools (`drizzle-kit`'s bundled esbuild). | Production dependencies audit clean. Don't run `npm audit fix -force`: it upgrades Next.js across a major version. |
| `npm` warns about install scripts | npm's install-script safety feature. | Harmless. To silence it: `npm install-scripts approve esbuild`. |

### Clearing rate-limit counters (development only)

```
docker compose exec redis sh -c \
  "redis-cli --scan --pattern 'rl:*'    | xargs -r redis-cli del; \
   redis-cli --scan --pattern 'lock:*'  | xargs -r redis-cli del"
```

The browser tests clear these automatically before each run, but only when `APP_ENV=development`.

CHAPTER 12

# Reference

## 12.1 HTTP API

All endpoints are under `/api` and exchange JSON. Every `POST`, `PATCH` and `DELETE` must come from the same origin; cross-site requests get `403 bad_origin`. Errors have the shape `{"error": code, "message": text}`. Responses are never cached.

**Table 12.1:** API endpoints

| Method | Path | Purpose (auth required) |
|---|---|---|
| **Accounts and sign-in** | | |
| POST | /api/auth/signup | Email sign-up; handle optional (none) |
| POST | /api/auth/verify-email | Confirm the emailed link (none) |
| GET | /api/auth/handle | Suggest a free handle; ?check= tests one (none) |
| POST | /api/auth/login | Password sign-in (none) |
| POST | /api/auth/logout | Sign out this session |
| POST | /api/auth/mfa/totp | Complete MFA, or step up, with an authenticator code |
| POST | /api/auth/totp/setup | Start authenticator enrolment (QR code and key) |
| POST | /api/auth/totp/confirm | Confirm enrolment; returns recovery codes on the first factor |
| POST | /api/auth/passkey/register-options | Begin passkey registration |
| POST | /api/auth/passkey/register-verify | Finish passkey registration |
| POST | /api/auth/passkey/login-options | Begin a passkey ceremony (sign-in, MFA or step-up) |
| POST | /api/auth/passkey/login-verify | Finish the ceremony |
| DELETE | /api/auth/passkey/[id] | Remove a passkey (step-up; not the last factor) |
| POST | /api/auth/recovery-codes | Generate new recovery codes (step-up) |
| POST | /api/auth/recovery/start | Use a recovery code; starts the 24-hour cooldown |
| POST | /api/auth/recovery/complete | Finish recovery after the cooldown |
| POST | /api/auth/recovery/cancel | Cancel an open recovery |
| GET | /api/auth/sessions | List signed-in devices |
| DELETE | /api/auth/sessions/[id] | Sign out a device |
| POST | /api/auth/password | Change password |
| GET | /api/me | Profile, factors, owner checklist |
| PATCH | /api/me/settings | Audit retention (1–24 months) and read receipts |
| GET | /api/me/export | Download all my data (step-up) |
| POST | /api/me/erase | Erase my account (step-up; confirmation phrase) |
| **Numbers and tokens (owner)** | | |

| Method | Path | Purpose (auth required) |
|---|---|---|
| GET, POST | /api/numbers | List or add protected numbers; POST takes phone, country, label |
| DELETE | /api/numbers/[id] | Remove a number and its tokens (step-up) |
| GET | /api/tokens | List tokens; ?recommend=<context> returns a tier recommendation |
| POST | /api/tokens | Create a token; the response holds the link and short code (shown once) |
| POST | /api/tokens/[id] | pause, resume, revoke, rotate, extend, clone |
| POST | /api/panic | Revoke every token (no step-up) |
| GET | /api/dashboard | Stats, tokens and recent feed |
| **Decode gate and contact** | | |
| POST | /api/gate/start | Start, or resume, a gate request for a code |
| GET | /api/gate/[id] | Current state of a request |
| POST | /api/gate/[id]/otp | send or verify a one-time code |
| POST | /api/gate/[id]/consent | Send a Tier 3 request with an optional note |
| POST | /api/gate/[id]/outcome | Choose reveal, chat or app_call |
| GET | /api/reveal/[grantId] | Watermarked PNG, single use, 60 seconds |
| GET | /api/requests | Owner: Tier 3 requests |
| POST | /api/requests/[id]/decision | Owner: approve_relay, reveal_once, decline, decline_block |
| **In-app calls (over data)** | | |
| POST | /api/gate/[id]/outcome | With choice: app_call: start an in-app call; returns appCallId |
| GET | /api/calls/incoming | Is someone calling me? Polled every 3 s by the open app; also marks me online |
| GET | /api/calls/ice | Short-lived TURN credentials and iceTransportPolicy |
| GET | /api/calls/[id] | Call state: role, other handle, status, ring deadline, owner online, chat link |
| POST | /api/calls/[id] | action: accept or decline (owner), cancel or hangup (either side) |
| GET | /api/calls/[id]/signal | Take (and delete) waiting offer, answer and ICE messages, plus the call state; doubles as the heartbeat |
| POST | /api/calls/[id]/signal | Send offer, answer, ice or restart; non-relay candidates are dropped |
| **Chat, safety and moderation** | | |
| GET | /api/chats | My conversations, with unread counts |
| GET, POST | /api/chats/[id]/messages | Read (?after=) or send messages; GET also returns call (whether the caller can start a private call, and the request to use) |
| POST | /api/chats/[id]/block | Block the other person |
| POST | /api/reports | Report from a conversation or a request |
| GET | /api/moderation | Moderator: queue (?status=open\|actioned\|dismissed) |

| Method | Path | Purpose (auth required) |
|---|---|---|
| POST | /api/moderation/[id] | Moderator: dismiss, warn, suspend, close_conversation |
| GET, POST | /api/notifications | List notifications or mark them read; GET also returns chatsUnread |
| **Audit, evidence and administration** | | |
| GET | /api/audit | Owner feed (?before=<seq>) |
| GET | /api/audit/export | Evidence pack (?format=json\|pdf, from, to) |
| GET | /api/evidence/public-key | Ed25519 public key (none) |
| POST | /api/evidence/verify | Check an evidence pack's signature (none) |
| GET, POST | /api/admin/actions | Admin: list or request break-glass actions (step-up) |
| POST | /api/admin/actions/[id] | Admin: approve or reject (different admin; step-up) |
| GET | /api/dev/outbox | Development only: captured messages |

### 12.1.1 Gate response statuses

| | |
|---|---|
| ok | Includes requestId, tier, label, step (otp, consent, waiting, choose) and the allowed options. |
| unavailable | Identical for every reason: unknown, inactive, blocked, budget used, declined or expired. |
| limited | Includes retryAfter in seconds. |
| captcha_failed | The human check failed. |
| otp_wrong | The one-time code didn't match. |

### 12.1.2 Common error codes

| Code | Meaning |
|---|---|
| signin_required (401) | No valid session. |
| mfa_required (401) | The session is waiting for its second factor. |
| verification_required (403) | Email or phone not verified yet. |
| owner_setup_required (403) | Owner features need a second factor (and 18+ confirmation). |
| step_up_required (403) | A fresh second-factor check is needed; the interface handles it automatically. |
| rate_limited (429) | Throttled; see retryAfter or the Retry-After header. |
| email_unavailable (503) | The confirmation email couldn't be sent; signing up again sends a fresh link. |
| delivery_failed (503) | A one-time code couldn't be emailed; try again shortly. |

| Code | Meaning |
|---|---|
| `bad_origin` (403) | A cross-site request was blocked. |
| `invalid_input` (400) | Validation failed; the message names the field. |
| `invalid_phone`, `invalid_country` (400) | The number doesn't fit the selected country (the message names it), or the country code isn't recognised. A number starting with `+` or `00` overrides the country. |
| `handle_taken` (409) | Includes a fresh `suggestion`. |
| `invalid_handle` (400) | Handle breaks the rules (3–24 letters, numbers, _ or ., no `@`); includes a corrected `suggestion`. |
| `token_cap`, `number_cap`, `tier_unavailable` | 10 live tokens per number; 5 numbers; Tier 4 isn't available. |
| `invalid_step`, `already_revealed` | The gate option isn't available at this step; one reveal per request. |
| `already_decided`, `expired` | A consent request was already handled, or has expired. |
| `blocked`, `closed` | A chat is blocked, or closed by the owner or a moderator. |
| `already_in_call`, `not_answered`, `call_over` | One call at a time; media setup waits for the answer; the call has already ended. |
| `invalid_action`, `call_not_found` | Only the person being called can answer; the call doesn't exist or isn't yours. |
| `dual_control` | An administrator tried to approve their own request. |
| `last_factor`, `recovery_cooldown` | You can't remove the only second factor; recovery isn't finished yet. |

## 12.2 Pages

**Table 12.3:** Web pages

| Path | Who | What it does |
|---|---|---|
| `/` | Everyone | Introduction and a box to enter a link or code |
| `/signup`, `/login`, `/login/mfa` | Visitors | Registration (email or phone), sign-in, second factor and recovery |
| `/verify-email` | Visitors | Confirms the emailed link |
| `/onboarding` | New owners | Second factor, recovery codes, first number |
| `/t/[code]` | Callers | The decode gate |
| `/r/[id]` | Callers | Resume a request (linked from notifications) |
| `/dashboard` | Owners | Stats, tokens with actions, live audit feed, panic button |
| `/dashboard/tokens/new` | Owners | Create a token with a tier recommendation |
| `/dashboard/requests` | Owners | Tier 3 requests and decisions |

| Path | Who | What it does |
|---|---|---|
| `/dashboard/numbers` | Owners | Protected numbers |
| `/dashboard/audit` | Owners | Full audit trail; evidence downloads |
| `/chats`, `/chats/[id]` | Signed in | Conversations; block and report |
| `/report` | Signed in | Report a gate request |
| `/settings` | Signed in | Account, phone, factors, password, devices, privacy, notifications, export, delete |
| `/moderation` | Moderators | Report queue |
| `/admin` | Administrators | Dual-control actions |
| `/privacy` | Everyone | Plain-language privacy notice |
| `/dev/outbox` | Development | Captured emails |

## 12.3 Command-line scripts

**Table 12.4:** npm scripts

| Command | Purpose |
|---|---|
| `npm run dev` / `build` / `start` | Development server; production build; production server (port 3000) |
| `npm run keys:generate` | Write fresh secrets into `.env` (keeps existing ones unless `-force`) |
| `npm run db:migrate` | Apply SQL migrations, retrying while the database starts |
| `npm run db:generate` | Create a migration after changing `src/db/schema.ts` |
| `npm run seed` | Demo accounts and tokens (development only) |
| `npm run admin:create -- -handle=x -role=admin` | Grant a role from the server shell (audited) |
| `npm run jobs` / `jobs:once` | Housekeeping every 10 minutes / one pass |
| `npm run dev:totp - <key>` | Current authenticator code for a key |
| `npm run email:test -- -to=<address>` | Send one test email through the configured provider (Resend) |
| `npm test`, `test:unit`, `test:integration` | Vitest suites |
| `npm run test:e2e` | Playwright browser tests against a running app |
| `npm run attack:enumerate` | Enumeration campaign (refuses production configuration) |
| `npm run scan:plaintext` | Search the database and logs for phone numbers (`-path=`, `-logs=`) |
| `npm run audit:verify` | Verify the audit chain and anchor files |
| `npm run load:test` | Gate latency test (`-rps`, `-seconds`) |
| `npm run verify:all` | Every check above, with a summary |

| Command | Purpose |
|---|---|
| `npm run typecheck` / `lint` | TypeScript and ESLint |

## 12.4 Notifications

| Event | In-app | Email | To |
|---|---|---|---|
| Tier 3 request | ✓ | ✓ | Owner |
| Token frozen | ✓ | ✓ | Owner |
| Panic revoke used | ✓ | ✓ | Owner |
| Recovery started or completed | ✓ | ✓ | Account holder |
| Request accepted | ✓ | ✗ | Caller |
| Caller locked out | ✓ | ✗ | Caller |
| Moderation warning | ✓ | ✓ | Reported user |
| Report reviewed | ✓ | ✗ | Reporter |
| New chat started | ✓ | ✓ | Owner |
| Missed in-app call (no answer, busy, or cancelled while the app was closed) | ✓ | ✓ | Owner |
| Incoming in-app call | ring | ✗ | Owner, while the app is open |
| New chat message | ✓ | ✓ | Other party (at most every 15 min per chat; no message text) |

## 12.5 Glossary

| Term | Meaning |
|---|---|
| Token / code | A random identifier an owner shares instead of a phone number: a 16-character link code or a 10-character short code. |
| Decode gate | The checks a caller must pass before any outcome is delivered. |
| Tier | The protection level of a token (1 Open, 2 Gated, 3 Consent, 4 Vault in Phase 2). |
| Outcome | What the caller gets after the gate: a chat, an in-app call, or a reveal if the code has a number. |
| Watermarked reveal | The number drawn into an image stamped with the viewer's handle and the time, shown for 60 seconds. |
| Step-up | A fresh second-factor check before a sensitive action. |
| Passkey | A WebAuthn credential unlocked with the device's screen lock. Phishing-resistant. |

| Term | Meaning |
|---|---|
| TOTP | Six-digit codes from an authenticator app (RFC 6238). |
| Envelope encryption | Data encrypted with a per-user data key, which is itself encrypted (wrapped) by a key-encryption key (KEK). |
| Blind index | A keyed hash that allows equality lookups on encrypted values. |
| Pepper | A server-side secret mixed into a hash, here the token fingerprint. |
| Crypto-shredding | Making data unreadable by deleting its key. |
| Hash chain | A log in which each entry includes the hash of the previous one, so changes are detectable. |
| Anchor | A day's final chain hash, written to write-once storage. |
| Evidence pack | A signed export of an owner's audit trail for platforms or police. |
| Break-glass | Exceptional administrator actions that need two people (dual control). |
| E.164 | The international phone-number format, e.g. `+26876123456`. |
| OWASP ASVS | Application Security Verification Standard; Level 2 is the go-live bar. |

APPENDIX A

# Design decisions

These are the points where the specification left a choice open, or where the build departs from it. The full list, with reasons, is in `docs/ASSUMPTIONS.md`. Items marked **(review)** need the product owner's confirmation.

| Decision | Why |
|---|---|
| Drizzle instead of Prisma **(review)** | Prisma's migration engine is a binary that couldn't be downloaded in the build environment, so it couldn't be tested. Drizzle is pure TypeScript with plain SQL migrations on the same PostgreSQL. |
| Codes shown once **(review)** | Only HMAC fingerprints are stored, as rule 4 requires. A lost link is replaced by rotating the token. The alternative is storing codes encrypted, which would let owners copy them again. |
| `APP_ENV` separate from `NODE_ENV` | `next start` always sets `NODE_ENV=production`, even on a desktop. `APP_ENV` decides whether development stand-ins are allowed. |
| CAPTCHA before token lookup | A CAPTCHA failure after the lookup would reveal that the code exists. |
| Per-caller in-flight cap of 2 | Without it, a burst of parallel guesses was evaluated before the lockout applied, giving 99.90% blocked and failing the over-99.9% bar. With it: 99.95%. |
| Generated handles | Handles are visible to everyone a person contacts, so the default is random (`adjective_noun_NN`) and never derived from the email, name or phone. People can still type their own. |
| Chat by polling (3 s) | Works through any proxy with a stateless server. The message API is transport-independent, so WebSockets or server-sent events can be added later. |
| Region is country-level | Taken from the edge's country header, or the caller's phone country. No GeoIP database is shipped. |
| Panic without step-up | Revoking is always safe; re-issuing afterwards needs a fresh second factor. |
| Development outbox stores masked numbers | Even development databases pass the plaintext scan. |
| Resend for email, via its REST API | No SDK or SMTP dependency. Idempotent retries on 429 and 5xx only; text and HTML bodies; category tags; addresses never logged. The test sender is refused outside development. |
| Daily anchors for completed days only | A day's head is final only once the day ends. |
| Phase 2 seams | Tier 4 is in the schema but can't be selected; the KMS interface and the chat encryption module are isolated for later replacement. |

## Document history

| Version | Date | Change |
|---|---|---|
| 1.0 | 3 November 2026 | First edition, for Phase 1 (system version 0.1.0, including generated handles and the migration retry). |
| 1.1 | 4 November 2026 | Email through Resend; handle validation and autofill protection on sign-up; recovery from email failures. |
| 1.2 | 4 November 2026 | Country picker for every phone field; clearer chat and relay-call guidance (option cards, call steps, **Call privately** in chats); chat notifications and unread badge. |
| 1.3 | 4 November 2026 | In-app calls over data through a self-hosted TURN relay (coturn); missed calls; call timeline in chats; native-app contract; phone relay made optional. |
| 1.4 | 4 November 2026 | Website visual refresh; iOS and Android app (Expo) with bearer sessions, calls, QR scanning and the full owner set-up. |
| 1.5 | 5 November 2026 | Safe groups and 24-hour statuses: contact-details guard, contact codes on posts, invite links, App Links and Universal Links that open the gate with the code filled in. |
| 1.6 | 5 November 2026 | Phone numbers optional: app-only codes (message and in-app call), two-step sign-in only for adding a number to show; phone-network calls removed. |
| 1.7 | 6 November 2026 | Email only: phone sign-up and all text messages removed; Tier 2 and 3 codes by email; sign-up returns people to the code or invite they opened. |
| 1.8 | 6 November 2026 | Joining a group needs an admin's approval; group search; codes shared in chats; 24-hour and one-person codes anywhere. |
| 1.9 | 6 November 2026 | New look for the website and app (Talk Sense style); screenshots retaken. |
| 1.10 | 6 November 2026 | Icon buttons: paper-plane send, flag, block, bin and sign-out icons; round call controls. |
| 1.11 | 6 November 2026 | Site address (`APP_URL`) moved to the top of `.env`; `RP_ID` follows it; production refuses a localhost address. |
| 1.12 | 6 November 2026 | Live address set: `https://pryingeyes.idealsoftwaresolutions.com`; the settings template, call relay and app store builds point at it. |
| 1.13 | 6 November 2026 | Self-contained website: built-in PostgreSQL (PGlite) and cache, self-preparing `.env`, migrations and jobs on start; `ADMIN_EMAILS`; deployment without Docker. |
| 1.14 | 6 November 2026 | Production Docker setup (`docker-compose.prod.yml`: website with its data in a volume, plus the relay); key fingerprint check; container-safe database lock; `/api/health`. |